TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

SAGARMATHA ENGINEERING COLLEGE

A

PROJECT REPORT

ON

YOGA ASSISTANT : A SYSTEM FOR YOGA POSE DETECTION AND CORRECTION

BY

INAPH KHADKA 33913

SAMYOG BABU ACHARYA 33926

SRIJANA THARU 33930

SWECCHYA KHANAL 33936

A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR IN COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

SANEPA, LALITPUR, NEPAL

APRIL, 2025

YOGA ASSISTANT : A SYSTEM FOR YOGA POSE DETECTION AND CORRECTION

BY

Inaph Khadka 33913

Samyog Babu Acharya 33926

Srijana Tharu 33930

Swecchya Khanal 33936

Project Supervisor

Er. Bipin Thapa Magar

Senior Lecturer

A project submitted to the Department of Electronics and Computer Engineering in partial fulfilment of the requirements for the degree of Bachelor in Computer Engineering

Department of Electronics and Computer Engineering

Institute of Engineering, Sagarmatha Engineering College

Tribhuvan University Affiliate

Sanepa, Lalitpur, Nepal

COPYRIGHT ©

DECLARATION

We hereby declare that the report of the project work entitled " Yoga Assistant: A System for Yoga Pose Detection and Correction" which is being submitted to the Sagarmatha Engineering College, IOE, Tribhuvan University, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Engineering, is a bonafide report of the work carried out by us. The material contained in this report has not been submitted to any University or Institution for the award of any degree.

Inaph Khadka 33913

Samyog Babu Acharya 33926

Srijana Tharu 33930

Swecchya Khanal 33936

CERTIFICATE OF APPROVALS

The undersigned certify that they have read and recommended to the Department of Electronics and Computer Engineering for acceptance, a project entitled **"Yoga Assistant: A System for Yoga Pose Detection and Correction"**, submitted by **Inaph Khadka, Samyog Babu Acharya, Srijana Tharu and Swecchya Khanal** in partial fulfillment of the requirement for the degree of "Bachelor in Computer Engineering".

.....

Supervisor:

Er. Bipin Thapa Magar

Department of Electronics and Computer Engineering,

Sagarmatha Engineering College

.....

External Examiner:

Dr.Dibakar Raj Pant

Professor

Pulchowk Campus

DEPARTMENTAL ACCEPTANCE

The project work entitled **" Yoga Assistant: A System for Yoga Pose Detection and Correction'**, submitted by **Inaph Khadka, Samyog Babu Acharya, Srijana Tharu and Swecchya Khanal** in partial fulfillment of the requirement for the award of the degree of **"Bachelor of Engineering in Computer Engineering"** has been accepted as a bonafide record of work independently carried out by team in the department.

.....

Bharat Bhatta

Head of Department

Department of Electronics and Computer Engineering,

Sagarmatha Engineering College,

Tribhuvan University Affiliate,

Sanepa, Lalitpur

Nepal.

ACKNOWLEDGEMENT

We express our sincere gratitude to all those who have supported us in the preparation of this report. First and foremost, we would like to extend our heartfelt thanks to our supervisor, Er. Bipin Thapa Magar, for providing invaluable guidance throughout the planning stages of our project. His insights and suggestions have significantly enriched our proposal. We also extend our appreciation to all the faculty members of the Department of Electronics and Computer Engineering for sharing their knowledge with us, enabling us to effectively prepare this report. Furthermore, we are thankful to our seniors and friends who provided valuable insights and references, enriching the content and relevance of this proposal.

ABSTRACT

The rapid advancements in computer vision and machine learning technologies have led to innovative applications in health and fitness. This project focuses on developing a yoga pose detection and correction system that leverages these technologies to improve online yoga sessions. The primary purpose of this system is to provide accurate posture analysis and feedback for users practicing yoga, helping them perform poses correctly and safely. The system is trained on a comprehensive dataset of 62,500 images from Kaggle, Google Dataset Search, and GitHub, which includes seven different yoga poses: Cobra, Downward Dog, Goddess, Mountain, Plank-Up, Tree, and Warrior. Data augmentation techniques such as flipping, brightness adjustment, and rotation were applied to increase the dataset size to 8,000–10,000 images per pose, ensuring better model generalization across various real-world conditions.

The system utilizes MediaPipe for real-time keypoint detection, capturing body points such as shoulders, elbows, and knees. Pose classification is then performed using a Random Forest Classifier (RFC) algorithm, which computes joint angles and compares them to ideal reference poses. A margin of $\pm 20^{\circ}$ is allowed for flexibility in the classification process to accommodate individual user variations. The results of the project demonstrate that the system can accurately classify poses and provide corrective feedback, ensuring that users receive useful guidance for improving their yoga practice.

The developed yoga pose detection and correction system effectively combines computer vision and machine learning to deliver precise posture feedback. This system can significantly enhance the quality of online yoga sessions, offering a scalable and adaptable solution for individuals seeking to practice yoga safely and effectively at home. Further work can focus on expanding the dataset, improving the correction mechanisms, and incorporating additional features such as personalized recommendations based on user progress.

TABLE OF CONTENTS

List of Figures

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AUC	Area Under the Curve
BGR	Blue Green Red (Color Format)
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
FPR	False Positive Rate
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HSV	Hue Saturation Value (Color Model)
IDE	Integrated Development Environment
JS	JavaScript
ML	Machine Learning
RAM	Random Access Memory
RFC	Random Forest Classifier
ROC	Receiver Operating Characteristics
SDLC	Software Development Life Cycle
TPR	True Positive Rate
TTS	Text-to-Speech
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Background

The Yoga Assistant project is driven by the growing demand for accessible, personalized online fitness solutions and the challenges in delivering effective real-time guidance for yoga practitioners remotely. Traditional yoga requires the expertise of an instructor to correct posture and ensure proper form, but as virtual platforms become more popular, there is a significant gap in providing real-time, personalized feedback for home practitioners. The advancement of computer vision and machine learning technologies offers an opportunity to address this gap. Tools like Google MediaPipe and machine learning algorithms such as Random Forest Classification can analyze body poses, but challenges remain in ensuring precise and efficient pose detection across varied body types, flexibility levels, and execution speeds. Additionally, costly motion capture systems limit accessibility for many users. The Yoga Assistant project detects and corrects yoga poses, offering users personalized feedback to prevent injuries, improve their practice, and enhance their overall yoga experience. The motivation behind this project is to make yoga more accessible and effective for those unable to attend in-person classes, providing a scalable solution for home practice, fitness tracking, and virtual coaching.

1.2 Problem Definition

The transition of yoga practice to online platforms has introduced significant challenges, primarily due to the lack of real-time feedback on posture and technique. Practicing yoga without proper guidance increases the likelihood of mistakes, which may lead to ineffective sessions or even injuries. This gap in personalized, corrective feedback diminishes the quality of the online yoga experience, leaving practitioners without the support they need to improve safely and effectively. [1]

To address this issue, we propose the integration of artificial intelligence into

online yoga platforms. This system will utilize real-time pose analysis to identify mistakes and provide corrective suggestions during practice. By leveraging available technology, the solution aims to enhance the safety, accessibility, and overall effectiveness of online yoga sessions, offering a seamless and engaging experience for practitioners of all levels. [2]

This system uses computer vision and machine learning to help users improve their yoga poses in real time. It tracks body keypoints using Google MediaPipe and compares the user's pose with ideal postures using a Random Forest Classifier (RFC). The system allows for a small range of flexibility ($\pm 20^{\circ}$) to account for natural differences in movement. If a user's pose is incorrect, it provides instant feedback with suggestions for improvement. This makes online yoga safer, more effective, and accessible, especially for those who do not have access to professional trainers or in-person classes. [3]

1.3 Objectives

The objectives of our project is to develop a yoga pose detection and correction system using RFC algorithm, ensuring accurate posture analysis and feedback for effective online yoga sessions.

1.4 Features

The features of our project are as follows:

1. Allow users to select the yoga pose they want to practice.
2. Track and identify keypoints of the human body in real-time.
3. Identify yoga poses from the detected keypoints.
4. Provide assistance for correcting user poses.

1.5 Feasibility

1.5.1 Technical Feasibility

The technical feasibility of the Yoga Assistant project is supported by the integration of mature technologies for dataset preparation, pose detection, and real-time

feedback. The project uses a dataset of 62,500 images from sources like Kaggle and GitHub, with data augmentation techniques such as flipping and rotation expanding it to 8,000–10,000 images per pose. TensorFlow and OpenCV facilitate efficient image manipulation and processing. Google MediaPipe is employed for real-time body keypoint detection, while TensorFlow powers pose recognition with the Random Forest Classifier (RFC) for accurate pose classification. These frameworks ensure scalability, with easy model retraining and expansion, backed by extensive open-source resources. This combination ensures the system delivers reliable, real-time feedback while supporting continuous improvements.

1.5.2 Operational Feasibility

The system is designed with simplicity in mind, requiring only a webcam or smartphone camera to function. Users can select from yoga poses like Cobra, Downward Dog, Warrior, Tree Pose , Mountain, Goddess, Plank up and receive real-time guidance to improve their form. This makes the system easy to incorporate into daily routines without the need for special equipment. Furthermore, the system is scalable, with the potential to add more poses or other fitness activities in the future. Its compatibility with various devices ensures that a wide range of users can access and benefit from it.

1.5.3 Economic Feasibility

The Yoga Assistant project is cost-effective for both developers and users. For developers, the use of open-source tools like TensorFlow, OpenCV, and Google MediaPipe significantly reduces initial development costs, making it an affordable project to build and maintain. For users, the system offers a low-cost solution to personalized yoga practice, as it eliminates the need for in-person sessions or expensive equipment.

1.6 System Requirements

The system requirements of our project are:

1.6.1 Software Requirements

The software requirements for our project are as follows:

(a) FrontEnd and BackEnd :HTML, CSS, JS, Django

(b) IDEs such as Visual Studio Code

(c) Libraries of Python: Numpy, Pandas, OpenCV, MediaPipe

(d) Libraries of Deep Learning : Tensorflow

1.6.2 Hardware Requirements

The hardware requirements for our project are as follows:

(a) To run the application:
Minimum of 8GB RAM, Intel Core i5 or equivalent to ensure smooth operation of the application.

(b) To train the model:

S.N	Particulars	System Specification
1	Total RAM	16 GB
2	Used RAM	8 GB (Approx.)
3	Total Disk Space	512 GB SSD
4	Used Disk Space	150 GB (for models, datasets, and logs)
5	GPU Used	NVIDIA RTX 3060 (6 GB VRAM)
6	CPU Used	Intel Core i7-12700K
7	Operating System	Ubuntu 20.04 / Windows 11
8	Frameworks Used	TensorFlow 2.x, OpenCV, Mediapipe
9	Python Version	3.9
10	IDE/Environment Used	Jupyter Notebook / VS Code
11	Network Requirements	10 Mbps (for downloading datasets/models)

Table 1.1: System Specification Table for Yoga Pose Detection System

1.6.3 Functional Requirements

(a) User Interface: Design an intuitive interface for users to select poses and receive feedback.

(b) KeyPoint Detection: Implement a real-time algorithm to detect key body joints.

(c) Pose Recognition: The system must accurately recognize various yoga poses based on detected body keypoints.

(d) Correction Guidance: Algorithms to analyze and correct user's poses.

1.6.4 Non-functional Requirements

(a) Performance: The system should respond to user interactions and provide feedback with minimal latency, ensuring a smooth and responsive user experience.

(b) Accuracy: The pose recognition and correction algorithms should achieve a high level of accuracy in detecting poses and providing guidance to users.

(c) Scalability: The system should be able to handle a growing number of users and concurrent sessions without significant degradation in performance.

(d) Compatibility: The application should be compatible with a variety of devices and operating systems, ensuring a seamless experience for users regardless of their preferred platform.

CHAPTER 2
LITERATURE REVIEW

Yoga is an excellent medium of physical exercise, offering numerous health benefits when practiced correctly. However, improper execution of yoga poses can lead to adverse effects. To ensure safe practice, having a trainer to guide and monitor the accuracy of different yoga poses is important. To address this, a system is proposed that aims to assist yoga enthusiasts in performing various poses correctly and validate their form. By integrating computer vision techniques, the system analyzes the user's pose and provides guidance to correct it based on yoga domain knowledge. [4]

The system uses pose estimation algorithms to detect key points on the body, and technologies like OpenCV and MediaPipe are incorporated for this purpose. OpenCV, an open-source library for computer vision, is used for image and video processing, enabling real-time tracking and manipulation of data. MediaPipe, a framework for building multimodal processing pipelines, is particularly used for detecting and tracking human body keypoints. These technologies are effective for detecting poses because they can process real-time video inputs and recognize complex human movements. The integration of these libraries is a good technical choice given their robustness and wide support in the community. [5]

The system's pose recognition mechanism uses a model trained with a dataset of various yoga poses. The model relies on machine learning algorithms, such as a Random Forest Classifier (RFC), to classify the poses and compare the detected body keypoints with the ideal pose standards. The use of RFC is a valid approach for pose classification as it can handle multiple features and identify key differences between poses, making it a suitable choice for yoga pose recognition. The feedback is based on the joint angles calculated from the detected keypoints, and the system offers suggestions for corrections if the pose deviates beyond a specified threshold. This real-time feedback mechanism is an essential feature of the system as it helps users maintain proper posture. [6]

OpenCV is an open-source library widely used for computer vision, image processing, and machine learning. It excels in processing real-time data, allowing us to manipulate images and videos so that algorithms can identify objects such as cars, traffic signals, number plates, faces, and even human handwriting. When combined with other data analysis libraries, OpenCV can process images and videos to meet specific requirements and achieve desired outcomes.Mediapipe is a framework mainly used for building multimodal audio, video, or any time series data. With the help of the MediaPipe framework, an impressive ML pipeline can be built for instance of inference models like TensorFlow, TFLite, and also for media processing functions. [7]

One strength of the system is its ability to provide real-time feedback, which is highly valuable for users practicing yoga at home. The integration of computer vision techniques with machine learning for pose recognition and correction makes the system highly functional and user-friendly. The real-time processing of pose data ensures that users receive prompt guidance, improving the accuracy of their practice. [8]

Various techniques have been proposed to classify yoga positions, particularly the sun salutation sequence. J. Palanimeera . utilized classifiers such as Logistic Regression, SVM, Naive Bayes, and KNN, with KNN achieving 96 percentage accuracy. Kishore . employed deep learning architectures including EpipolarPose, OpenPose, PoseNet, MoveNet, and MediaPipe, with MediaPipe achieving the highest estimation accuracy and MoveNet being the fastest. Rishan developed a vision-based system using OpenPose with 99.91 percentage accuracy for six asanas. [8]

However, while the use of RFC for pose recognition is a reasonable choice, there could be room for improvement in the model's accuracy and efficiency. The system's accuracy, reported to be around 96.08%, is quite good, but accuracy can fluctuate due to variations in body shape, lighting, and environmental conditions. To improve performance, additional training data could be used to account for more diverse user poses, body types, and environmental variables. The dataset could be expanded to include more variations of poses under different lighting conditions, clothing types, and angles. Moreover, incorporating deep learning models such as

Convolutional Neural Networks (CNNs) or PoseNet, which are specifically designed for image-based tasks, could further enhance the system's ability to detect poses more accurately, even in challenging conditions. [9]

Additionally, the system's performance can be influenced by hardware limitations. Since the pose detection and feedback rely on video input, the quality of the camera or device used for input could affect the accuracy and latency of pose detection. This means that ensuring compatibility with a variety of devices, especially low-cost options like smartphones, should be prioritized.

Overall, the system represents a strong step toward making yoga practice more accessible and safe. The integration of computer vision, machine learning, and real-time feedback demonstrates solid technical feasibility. However, future improvements could focus on refining the pose recognition algorithms, expanding the dataset for more comprehensive pose coverage, and enhancing the system's performance under diverse conditions. [10]

2.1 Comparative Study: Our Project vs. Existing Work

To strengthen the foundation of our project, we conducted a comparative analysis with a similar research titled *"Real-Time Yoga Pose Detection using Machine Learning Algorithm"* by Jothika Sunney. While both systems aim to assist yoga practitioners using computer vision and machine learning, key differences in approach, implementation, and outcome highlight the advantages of our proposed system. [11]

Table 2.1: Comparison between Our Project and Existing Work

Aspect	**Our Project (Yoga Assistant)**	**Jothika Sunney's Project**
Dataset Size & Variety	62,500+ images augmented to 8,000–10,000 per pose (7 poses)	1,551 images covering 5 poses
Pose Detection Method	MediaPipe + Random Forest Classifier + Joint Angle Calculation	MediaPipe BlazePose + XGBoost Classifier
Pose Correction Mechanism	Real-time correction using joint angles and Mean Absolute Error (MAE)	Only classification with feedback based on prediction confidence
Feedback Mechanism	Real-time audio feedback using Text-to-Speech (TTS)	Visual feedback via OpenCV window
System Design	Modular and scalable with web interface using Django framework	Implemented in Jupyter Notebook without user interface
Performance & Flexibility	High classification accuracy with support for $\pm 20^{\circ}$ angle variation	Focuses on classification accuracy with 8ms latency, no correction system
Future Scope	Personalized feedback, AR integration, wearable support, multi-user capability	Mobile integration suggested; limited future roadmap

This comparison demonstrates that our project not only classifies yoga poses but also provides real-time correction and assistance using joint angle analysis. By integrating a feedback mechanism via speech, the system becomes more intuitive and hands-free, making it highly suitable for home practitioners. Moreover, the larger dataset, robust system architecture, and forward-looking features make our Yoga Assistant a more comprehensive and scalable solution compared to the existing work.

CHAPTER 3
RELATED THEORY

3.1 HTML

HTML, which stands for Hypertext Markup Language, is the foundational language for creating content on the World Wide Web. It is a markup language that structures the content on a web page using a system of elements and tags. These elements define the different parts of a web page, such as headings, paragraphs, links, images, and more. [12]

In addition,Semantic HTML is a fundamental principle in website development which holds importance in crafting web pages with clarity, accessibility, and meaningful structure.The adoption of semantic HTML uplifts a website's usability, and future adaptability in the dynamic landscape of the digital realm. [13]

3.2 CSS

Cascading Style Sheets is a style sheet language used to describe the presentation of a document written in HTML. CSS allows web designers to separate the presentation of a document from its content, making it easier to update and maintain the appearance of a website.

CSS works by defining rules that specify how HTML elements should be displayed on a page, such as the color, font, size,and position of text and images.CSS uses a hierarchical structure known as the cascade to determine which style rules apply to a given element. This means that if multiple rules apply to the same element, CSS will apply the most specific and relevant rule based on the order of importance and specificity. Additionally, CSS also supports the concept of inheritance, which means that styles applied to a parent element will be inherited by its child elements unless explicitly overridden. [14]

3.3 JavaScript

JavaScript is a high-level programming language used to develop interactive web pages and web apps. It enables web developers to include dynamic and interactive components such as animations, user input forms, and real-time updates on their websites. JavaScript is one of the key technologies used in web development and is supported by all modern web browsers.
It is an object-oriented language based on prototypes, which means that objects can inherit properties and methods from other objects. It also has functions, which are reusable sections of code that may be called and run at any moment. JavaScript code is executed on the client side, which means it is executed by the user's browser rather than the server. This enables fast and responsive user interfaces while also reducing server burden. [15]

3.4 Django

Django is a high-level web framework for building web applications in Python. Developed to prioritize simplicity, flexibility, and scalability, Django follows the"don't repeat yourself" (DRY) and "convention over configuration" principles. It encourages rapid development by providing a clean and pragmatic design.One of Django's key features is its Object-Relational Mapping (ORM) system, which allows developers to interact with the database using Python code instead of SQL queries directly. This simplifies database-related tasks and promotes code readability. Additionally, Django includes an administrative panel that can be easily customized, providing an out-of-the-box solution for managing application data. [16]

3.5 Dataset

A dataset is a collection of data, typically organized in a tabular format with rows representing individual data points (samples) and columns representing features (attributes or variables) of the data. Each row corresponds to a single instance, such as a person, object, or event, and each column represents a specific piece of information about that instance.

Datasets are used in various fields for tasks such as training machine learning models, conducting statistical analysis, and testing hypothesis. They can be collected from various sources, including experiments, surveys, observations, and existing databases. Datasets can vary greatly in size, complexity, and format, depending on the specific application and the type of data being collected. [17]

3.6 Numpy

NumPy is a fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
NumPy is essential for numerical and mathematical operations in Python, especially for tasks like linear algebra, Fourier analysis, random number generation, and more. Its efficiency and speed make it a cornerstone of many scientific and data-related Python libraries and applications. [18]

3.7 Pandas

Pandas is a popular open-source Python library used for data manipulation and analysis. It provides easy-to-use data structures, such as Series (1-dimensional) and DataFrame (2-dimensional), that allow you to work with structured data effectively.
Pandas is widely used in data science, machine learning, and data analysis for tasks like reading and writing data in various formats (e.g., CSV, Excel), cleaning and transforming data, handling missing values, and performing statistical operations. Its intuitive and powerful features make it a valuable tool for working with structured data in Python. [19]

3.8 OpenCV

OpenCV is an open-source library for image processing and computer vision tasks. It offers tools for image and video processing, object detection, machine learning integration, and deep learning. With its performance optimization and extensive functionality, it is used to capture video from the webcam, process the video frames,

and show results on the screen. It sets up the webcam with proper settings like brightness and resolution. It flips the video so the user sees a mirror view, resizes frames to make processing faster, and displays the video with extra information like pose names and body landmarks. It handles all the video-related tasks in real time, making it an essential part of the pose detection system. It is widely used in both research and industry for various computer vision applications. [20]

3.9 MediaPipe

Mediapipe is a cross-platform library developed by Google that provides pre-built machine learning solutions for real-time tracking and analysis of visual data. Mediapipe is used for detecting and tracking human body landmarks, such as shoulders, elbows, hips, and knees, in the video stream. It identifies these key points and connects them to form a skeleton-like representation of the user's body, which is critical for pose detection. The library's Pose solution processes each video frame to extract 3D landmark coordinates, which are then used to predict the yoga pose and analyze body alignment. Mediapipe's efficiency and real-time performance make it ideal for this application, as it enables accurate pose detection and overlay of visual landmarks on the video feed. Its ability to handle complex tasks with minimal coding simplifies the integration of machine learning models into the system. [21]

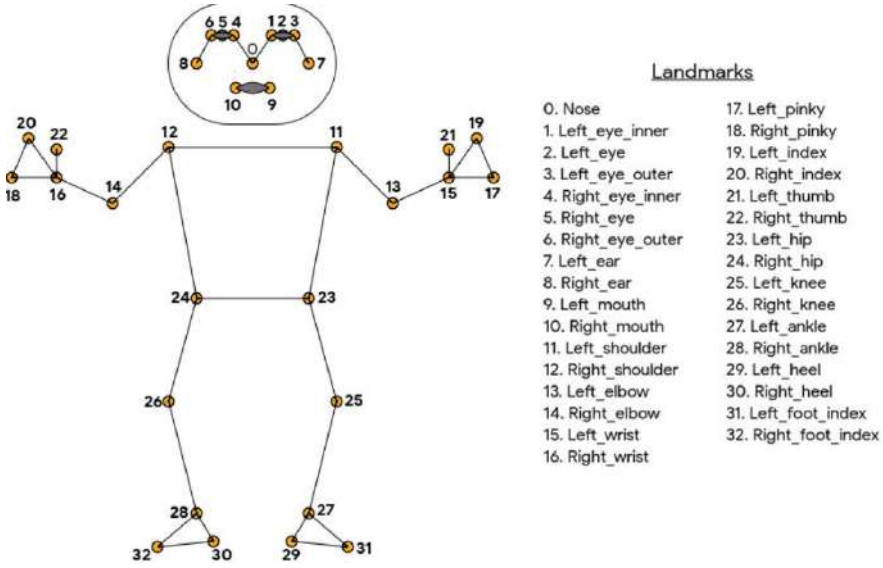


Figure 3.1: Landmarks

3.10 RFC

Random Forest Classifier (RFC) is a type of machine learning model that makes predictions by combining multiple decision trees. Each tree looks at the data in a different way, and the final prediction is based on the majority vote from all the trees. The RFC model is used to recognize different yoga poses based on the data from the body landmarks detected in the video. The model has already been trained on examples of poses and is loaded using Pickle to make predictions on new data. RFC is powerful because it can handle a lot of different data and still make accurate predictions, which is helpful for recognizing complex poses in real-time. [22]

3.11 Pickle

Pickle is a Python library used for serializing and deserializing data, meaning it converts Python objects into a format that can be saved to a file and later restored back into memory. It is used to load the model trained in dataset that recognizes different yoga poses. The model is saved in a file, and using pickle.load(), it is deserialized and brought back into the program for use. This allows the system to use a trained model without needing to retrain it every time the program runs. Pickle simplifies the process of saving machine learning models or any other complex data structures and efficiently reusing them in future sessions, ensuring that the system can quickly make predictions on pose detection. [23]

3.12 Pyttsx4

Pyttsx4 is a Python library designed for converting text to speech (TTS) and operates offline by leveraging the system's built-in speech engines, such as SAPI5 on Windows or NSSpeechSynthesizer on macOS. It enables real-time speech synthesis, making it suitable for applications requiring audio feedback, like virtual assistants, educational tools, and accessibility software. Pyttsx4 allows customization of speech properties, including voice selection, volume, pitch, and speech rate, offering a personalized and versatile TTS experience. The library initializes a TTS engine through pyttsx4.init(), processes text with say(), and plays synthesized speech

with runAndWait(). As it works offline, pyttsx4 is ideal for secure environments where internet connectivity is not an option. Its simplicity and cross-platform support make it a practical choice for developers adding audio capabilities to Python applications. [24]

3.13 Multiprocessing

Multiprocessing is a Python library that allows programs to run multiple tasks simultaneously by creating separate processes. It is used to handle text-to-speech (TTS) functionality without slowing down the real-time pose detection and video processing. By running the TTS in a separate process, the system ensures that audio feedback is generated and spoken while the main program continues to analyze video frames and detect poses. The library's JoinableQueue is used to pass messages (suggestions or feedback) from the main process to the TTS process, ensuring smooth communication between them. This design improves efficiency and responsiveness, allowing the system to provide real-time visual and audio feedback without interruptions or delays. Multiprocessing makes the program more robust and ensures both tasks pose detection and TTS work seamlessly together. [25]

CHAPTER 4
METHODOLOGY

4.1 Software Development Life Cycle

We have implemented our project using Agile model of Software Development life cycle(SDLC).This approach allowed us to implement and improve then system based on the different phases of the software development life cycle.

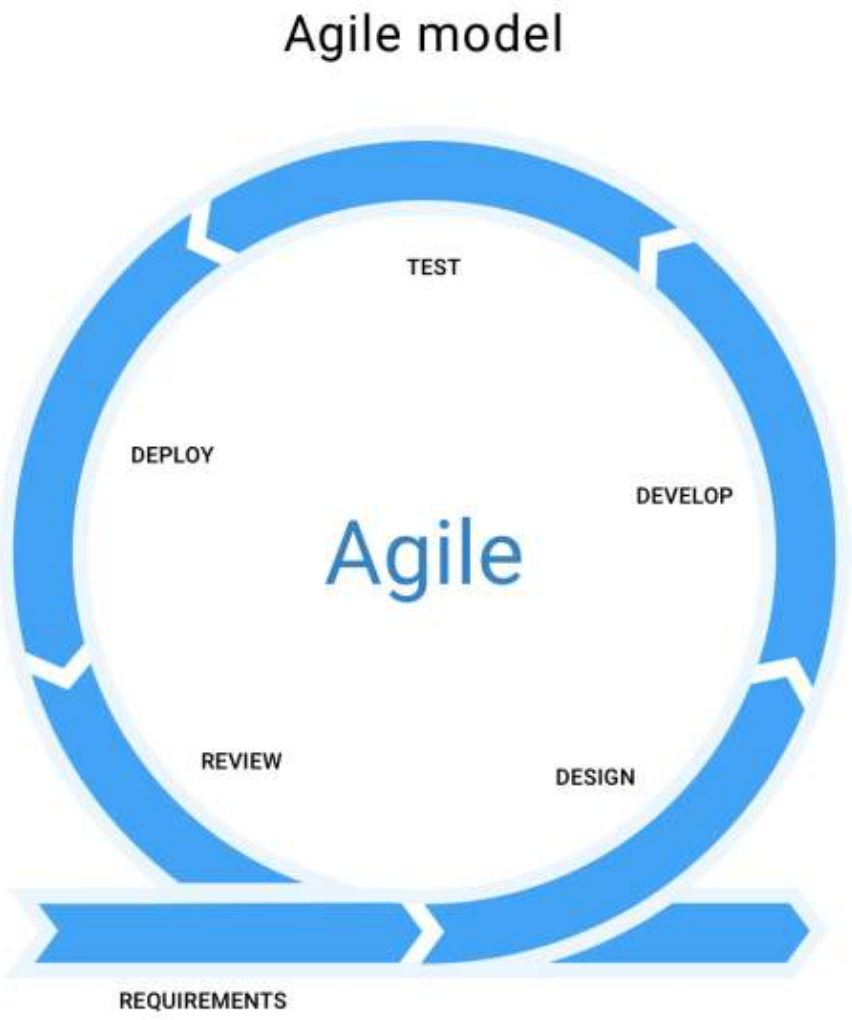


Figure 4.1: Agile Software Development Model

The Agile software development model is the most suitable approach for the Yoga Assistant project, as it enables flexibility, continuous improvement, and user-driven

enhancement which are critical factors for developing a yoga pose detection and correction guidance system. This project involves the integration of pose classification, and correction guidance, all of which require iterative refinements to ensure accuracy and effectiveness. Agile's iterative nature allows for continuous enhancements to the RFC algorithm, improving pose detection accuracy by incorporating real-world user feedback and refining the correction mechanisms over time.

During development, we encountered challenges related to pose misclassification, which required an iterative approach for improvement. Agile allowed us to rapidly collect additional training data, fine-tune the Random Forest Classifier's hyperparameters, and refine the way we computed joint angles for better pose recognition. Although MediaPipe provided a strong foundation, building a correction system for inaccurate poses was significantly more complex. In many cases, calculations appeared correct in theory but failed in real-world testing. Agile's iterative cycles enabled continuous testing and refinement, ensuring that the system delivered accurate feedback and adapted to unforeseen issues.

Moreover, the precision required for yoga pose analysis necessitates a development approach that supports early prototyping, rigorous testing, and frequent evaluation. Agile facilitates incremental improvements in pose classification, joint angle computation, and correction guidance, ensuring that the system adapts to diverse user variations and environmental conditions. Unlike traditional linear models, Agile enables rapid modifications based on insights from yoga practitioners and instructors, making the system more reliable and user-friendly. Additionally, given the evolving nature of yoga practices, Agile allows for seamless integration of new poses, refinement of correction techniques, and expansion of dataset coverage, ensuring the solution remains scalable and adaptable to different yoga styles.

4.2 System Block Diagram

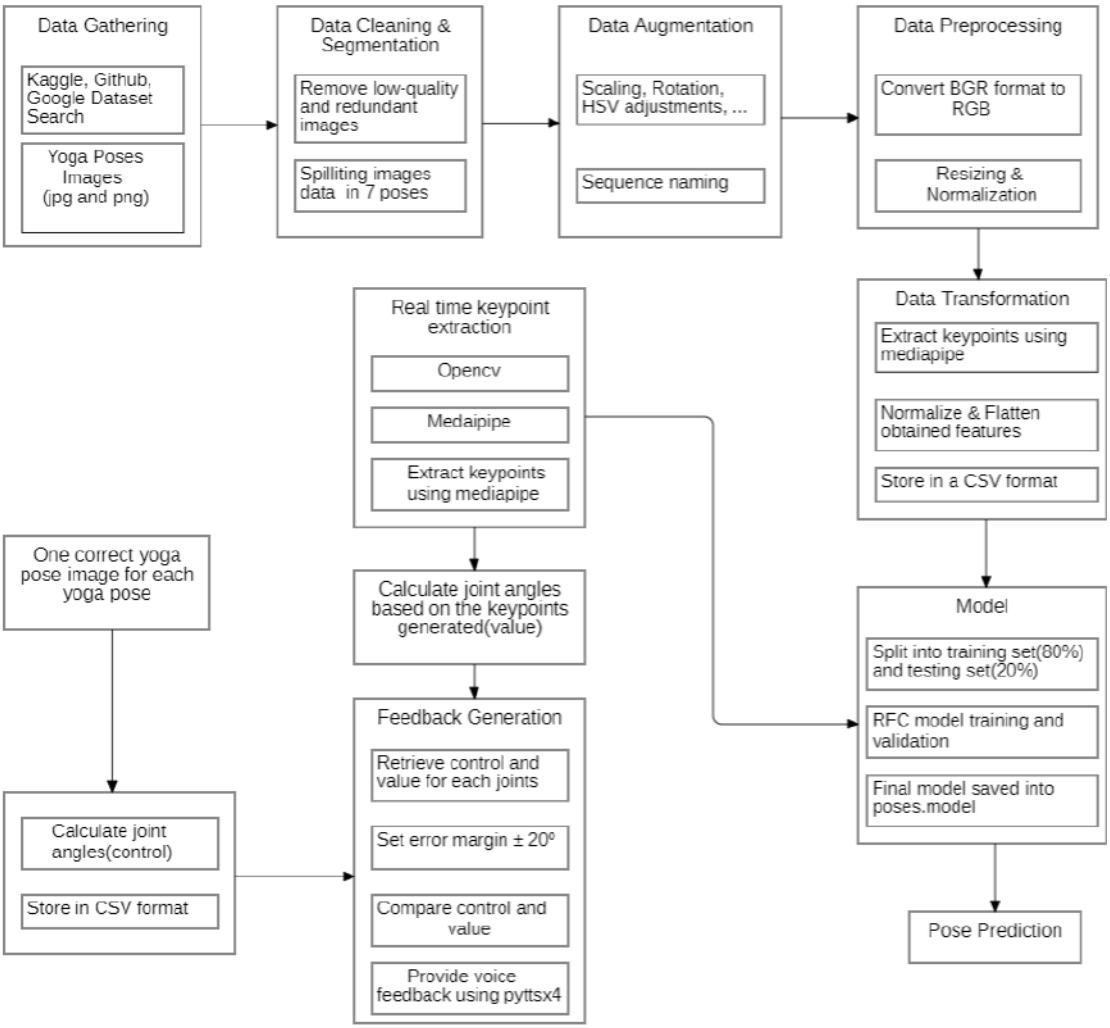


Figure 4.2: System Block Diagram of Yoga Assistant: A System for Yoga Pose Detection and Correction

The block diagram illustrates the end-to-end workflow of a yoga pose detection and correction system. The process begins with data gathering, where yoga pose images are gathered from online sources such as Kaggle, GitHub, and Google Dataset Search. These images undergo data cleaning to eliminate low-quality and redundant entries, followed by segmentation into seven distinct yoga poses: Cobra, Downward Dog, Goddess, Tree, Mountain, Plankup, and Warrior. To enhance the robustness of the dataset, data augmentation techniques such as scaling, rotation, and color space adjustments are applied. The images are then preprocessed through format conversion (BGR to RGB), resizing, and normalization. Using MediaPipe, key

landmarks are extracted from each image and converted into numerical features, which are normalized, flattened, and stored in CSV format. A Random Forest Classifier (RFC) is trained on this processed data, with an 80/20 train-test split, and the trained model is saved for deployment. In the real-time phase, the system leverages OpenCV and MediaPipe to extract body keypoints from live video input. These keypoints are used to compute joint angles, which are then compared against predefined control angles extracted from reference images for each pose. The control data is also stored in CSV format. An error margin of $\pm 20^{\circ}$ is set to assess the correctness of each joint's alignment. If the deviation exceeds this threshold, the system generates corrective feedback using a text-to-speech engine (pyttsx4), guiding the user to adjust their posture accordingly. Simultaneously, the trained model classifies the current pose, enabling both recognition and correction in a seamless, real-time environment.

4.3 Flowchart

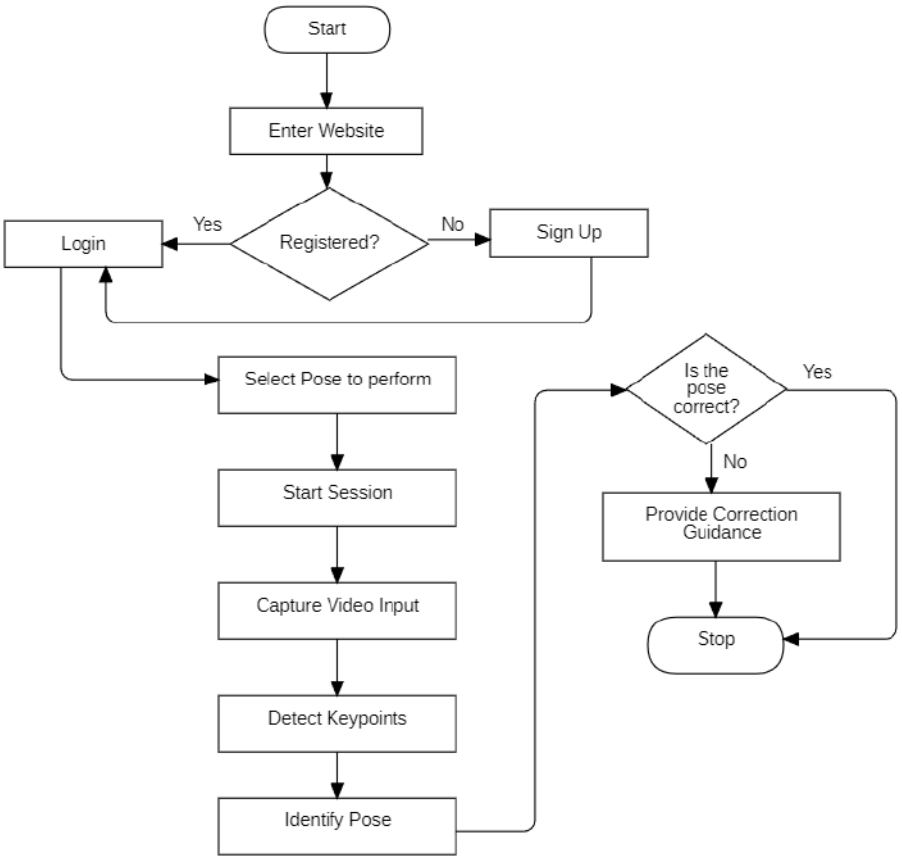


Figure 4.3: Flowchart of Yoga Assistant: A System for Yoga Pose Detection and Correction

The process begins when the user navigates to the website. The system checks if the user is registered on the website. If the user is registered, they proceed to log into their account; if not, they are directed to the sign-up page. After logging in, the user selects a pose they want to perform and is shown a preview of the selected pose. The user then starts a session to perform the pose, during which the system captures video input of the user. The system detects keypoints on the user's body from the video input and identifies the pose being performed. Based on the detected keypoints, the system provides guidance to the user on how to correct their pose if needed. Finally, the process ends.

4.4 Usecase Diagram

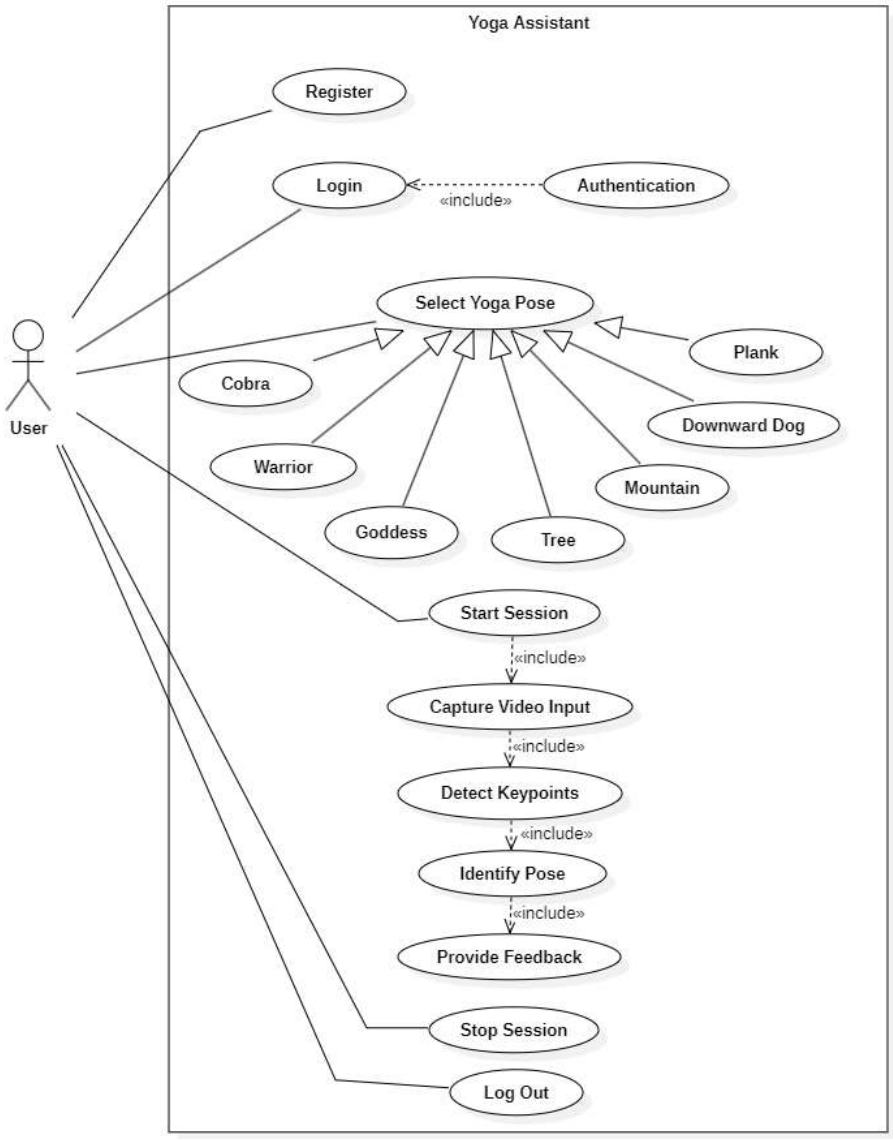


Figure 4.4: Usecase Diagram of Yoga Assistant: A System for Yoga Pose Detection and Correction

In this system, users engage by inputting personal details to establish their account or by logging in with existing credentials. Once logged in, they select from a range of predefined yoga poses. Upon selecting a pose, users activate the session with the "Start Session" button. Throughout the session, users adjust their posture according to real-time feedback provided by the system, which continuously captures video and detects keypoints on the user's body. This feedback aids users in refining their poses. When the session concludes, users halt the process by clicking the "Stop

Session" button. Behind the scenes, the system stores user details, verifies login credentials, loads pose information and keypoint data, initiates and terminates video capture and processing, identifies correct pose execution, and delivers real-time feedback to guide posture adjustments.

4.5 DFD 0

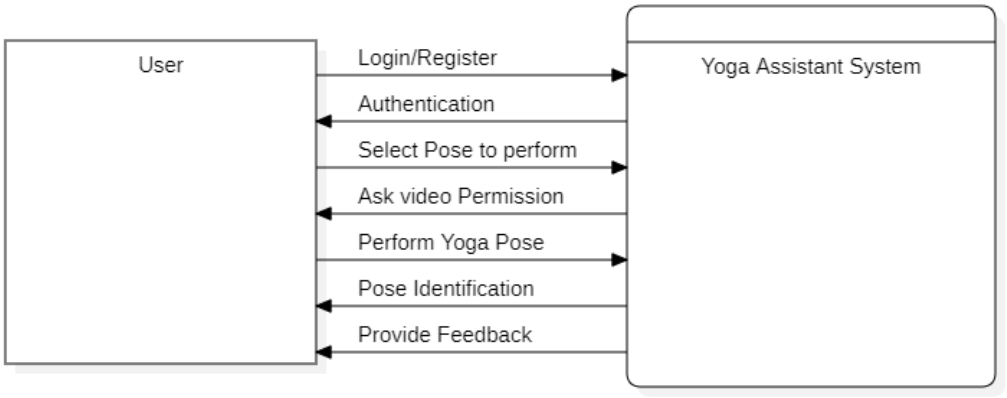


Figure 4.5: DFD-0 of Yoga Assistant: A System for Yoga Pose Detection and Correction

The flow diagram of a Yoga Assistant System, showing the interaction between the user and the system, and the internal processes involved. Initially, the user selects a yoga pose, at which point the system requests video permission to access the user's camera feed. The user then performs the selected yoga pose. The system processes this video input through a Pose Data Processing module, which detects key points on the user's body to identify the specific yoga pose being performed. Based on this identification, the system generates feedback to guide the user on improving their posture or confirm correct execution. This feedback is then provided to the user through the Yoga Assistant System, completing the interaction cycle.

4.6 Project Overview

4.6.1 Dataset Preparation

4.6.1.1 Collecting the Poses Dataset

To develop a robust yoga pose recognition and assistant system, we compiled a dataset of images representing various yoga poses, sourced from platforms such as Kaggle, Google Dataset Search, and GitHub. The dataset includes seven distinct poses: Cobra, Downward Dog, Goddess, Mountain, Plank-Up, Tree, and Warrior. For each pose, we initially collected between 800 and 1,000 images. To enhance the dataset's quality and usability, we conducted a thorough data cleaning process to remove redundant images and ensure consistency. Following this, we applied data augmentation techniques to expand the dataset, increasing the number of images to approximately 8,000 to 10,000 per pose.

During augmentation, the techniques and their corresponding parameters are detailed in Table 4.2.

This ensured a comprehensive representation of real-world scenarios, improving the model's ability to generalize effectively. Additionally, we aimed for a balanced dataset, maintaining an equal distribution of images across all yoga poses. A balanced dataset is critical for achieving optimal model performance, as it prevents bias towards any particular class and enhances the model's ability to accurately recognize and differentiate between poses.

Table 4.1: Distribution of the image dataset across various classes.

Pose Name	**Initial Image Count**	**Augmented Image Count**
Cobra (Class 0)	900	9,000
Downward Dog (Class 1)	950	9,500
Goddess (Class 2)	850	8,500
Mountain (Class 3)	1,000	10,000
Plank-Up (Class 4)	800	8,000
Tree (Class 5)	850	8,500
Warrior (Class 6)	900	9,000

Table 4.2: Augmentation Parameters for Image Dataset

Augmentation Technique	**Parameter**	**Value**
Horizontal Flip	Probability	50%
Shift, Scale, Rotate	Shift Limit	10%
	Scale Limit	20%
	Rotation	±30°
Random Brightness and Contrast	Brightness Adjustment	±20%
	Contrast Adjustment	±20%
HSV Adjustments	Hue	±10
	Saturation	±15
	Value	±10
Elastic Transform	Alpha	1
	Sigma	50
Coarse Dropout	Mask Regions	1 to 8
	Max Mask Size	30 pixels
Gaussian Blur	Kernel Size	3–7 pixels
Perspective Transform	Scale	5–10%

4.6.1.2 Preparing the Landmarks Dataset

For the preparation of landmarks dataset for individual poses, first of all, the images of each pose are converted to RGB format from their original BGR format. In the BGR format, each pixel of the image is represented by three color components: Blue (B), Green (G), and Red (R). For each pixel at position (i, j), the values are represented as

$$I_{\text{BGR}}(i, j) = [B(i, j), G(i, j), R(i, j)] \tag{4.1}$$

To convert the image to RGB format, the Blue and Red channels are swapped, while the Green channel remains unchanged. Mathematically, for each pixel, the transformation is:

$$I_{\text{RGB}}(i, j) = [R(i, j), G(i, j), B(i, j)] \tag{4.2}$$

where,
$R(i, j)$ is the original Red value of the pixel at position (i, j),
$G(i, j)$ is the original Green value of the pixel at position (i, j),
$B(i, j)$ is the original Blue value of the pixel at position (i, j).
This transformation is applied to every pixel in the image, resulting in a full image matrix in RGB format. After the conversion, the image can be processed for further tasks like landmark detection, which requires the RGB color format.
The detection of body landmarks from RGB images involves several steps, starting with the processing of the input image. Initially, the image is represented as a 3D matrix I_{RGB} of size $H \times W \times 3$, where H is the height, W is the width, and 3 corresponds to the Red, Green, and Blue color channels. Each pixel at position (i, j) in the image has corresponding values for the three color channels, denoted as:

$$I_{\text{RGB}}(i, j) = [R(i, j), G(i, j), B(i, j)] \tag{4.3}$$

The image undergoes preprocessing, which typically involves resizing and normalizing the pixel values. The resized image, with dimensions $H' \times W' \times 3$, has pixel

values normalized to the range [0, 1] by dividing each channel's intensity by 255:

$$I_{\text{normalized}}(i,j) = \frac{I_{\text{RGB}}(i,j)}{255} \tag{4.4}$$

This normalization ensures that the input values are within a consistent range, enabling the model to process them efficiently.

Next, a Convolutional Neural Network (CNN) is employed to extract hierarchical features from the image. The CNN applies convolution operations to the image, capturing spatial relationships that are essential for identifying body landmarks. Mathematically, the convolution operation at a pixel position (i, j) with a kernel K is given by:

$$F(i,j) = (I_{\text{normalized}} * K)(i,j) = \sum_{m,n} I_{\text{normalized}}(i+m, j+n) \cdot K(m,n) \tag{4.5}$$

where $*$ represents the convolution operation, K is the kernel (filter) applied to the image, and $F(i, j)$ is the feature map generated from the convolution.

Once the features are extracted, a deep learning model is used to detect the body landmarks. The model predicts the normalized coordinates (x_i, y_i) for each landmark in the range [0, 1], relative to the dimensions of the image. The predicted coordinates for the landmarks are expressed as:

$$L_{\text{predicted}} = [x_1, y_1, x_2, y_2, \ldots, x_n, y_n] \tag{4.6}$$

where n is the number of landmarks being detected.

Following the prediction, the normalized coordinates are rescaled to the original image dimensions. This re-scaling step involves multiplying the normalized coordinates by the width W and height H of the image, thus converting them back to pixel positions:

$$x_i = x_i' \times W, \quad y_i = y_i' \times H, \tag{4.7}$$

where x_i' and y_i' are the normalized predicted coordinates, and x_i and y_i are the corresponding pixel positions in the original image.

The final output consists of the pixel coordinates of the detected landmarks, which can be used for applications such as pose estimation, gesture recognition, or body

tracking. The detected landmarks are represented by the coordinates:

$$L_{\text{predicted}} = [x_1, y_1, x_2, y_2, \ldots, x_n, y_n] \tag{4.8}$$

which provide precise localization of key points on the human body.

After detecting the landmarks, the detected landmarks position are normalized to the image dimensions as [x,y,z,v].

where, x is the position of a landmark on the horizontal axis of the image,

y is the position of a landmark on the vertical axis of the image,

z is the depth position(how far is the landmark from the camera),

v is a confidence score indicating how well the landmark was detected.

x, y = pixel positions/image dimensions

If image width is 200px and landmark is 100px horizontally then,

x=100/200 = 0.5

Similarly, if image height is 400px and landmark is 100px vertically then,

y=100/400 = 0.25

z is calculated relative to the midpoint of the hips which is handled by mediapipe internally (z=0 at hips).

This process is iterated to collect the values of x,y,z,v for specific landmarks(nose, shoulders,hips,etc). Each body landmarks are combined into a single list which is also known as flattening data.

For any n*m, n*m matrix (2D array), flattening rearranges it into a single array (1D) by concatenating rows in order. Mathematically:

If the original matrix is:

$$A = \begin{array}{c|cccc} & 1 & 2 & \cdots & m \\ \hline 1 & a_{11} & a_{12} & \cdots & a_{1m} \\ 2 & a_{21} & a_{22} & \cdots & a_{2m} \\ 3 & a_{31} & a_{32} & \cdots & a_{3m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ n & a_{n1} & a_{n2} & \cdots & a_{nm} \end{array}$$

The flattened array B becomes:

$$B = \Big[a_{11}, a_{12}, \ldots, a_{1m}, a_{21}, a_{22}, \ldots, a_{2m}, a_{31}, a_{32}, \ldots, a_{3m}, \ldots, a_{n1}, a_{n2}, \ldots, a_{nm}\Big]$$

Finally it is then converted into a pandas DataFrame and saved it as a CSV file for the preparation of our final dataset containing the landmarks of each poses which are actually used for training and testing our pose classification model. This process is iterated for all the images in our dataset and our final dataset for pose classification model is prepared.

The final CSV file for the pose classification dataset typically stores the landmarks attributes in a tabular format. Each row corresponds to a single image (or pose), and the columns represent the attributes for all detected landmarks.

4.6.1.2.1 Format of the CSV File

Rows: Each row represents the flattened landmark data for one image or pose.

Columns: The last column contains a label for the pose, which is the target variable for classification, whereas the remaining columns store the flattened landmark attributes (x, y, z, v) for each landmark sequentially.

4.6.1.2.2 Attributes Stored

For each landmark detected, the following attributes are stored in sequence:

x: Normalized horizontal position of the landmark,

y: Normalized vertical position of the landmark,

z: Depth position relative to the reference point,

v: Confidence score for the landmark detection.

The following landmarks are detected for each pose, and each landmark has associated X, Y, and Z coordinates, which are not shown in this table for simplicity.

Table 4.3: Detected Landmarks

Nose	Left Shoulder	Right Shoulder	Left Elbow	Right Elbow
Left Wrist	Right Wrist	Left Hip	Right Hip	Left Knee
Right Knee	Left Ankle	Right Ankle	Left Heel	Right Heel
Left Foot Index	Right Foot Index			

4.6.2 Pose Classsification Model Architecture:

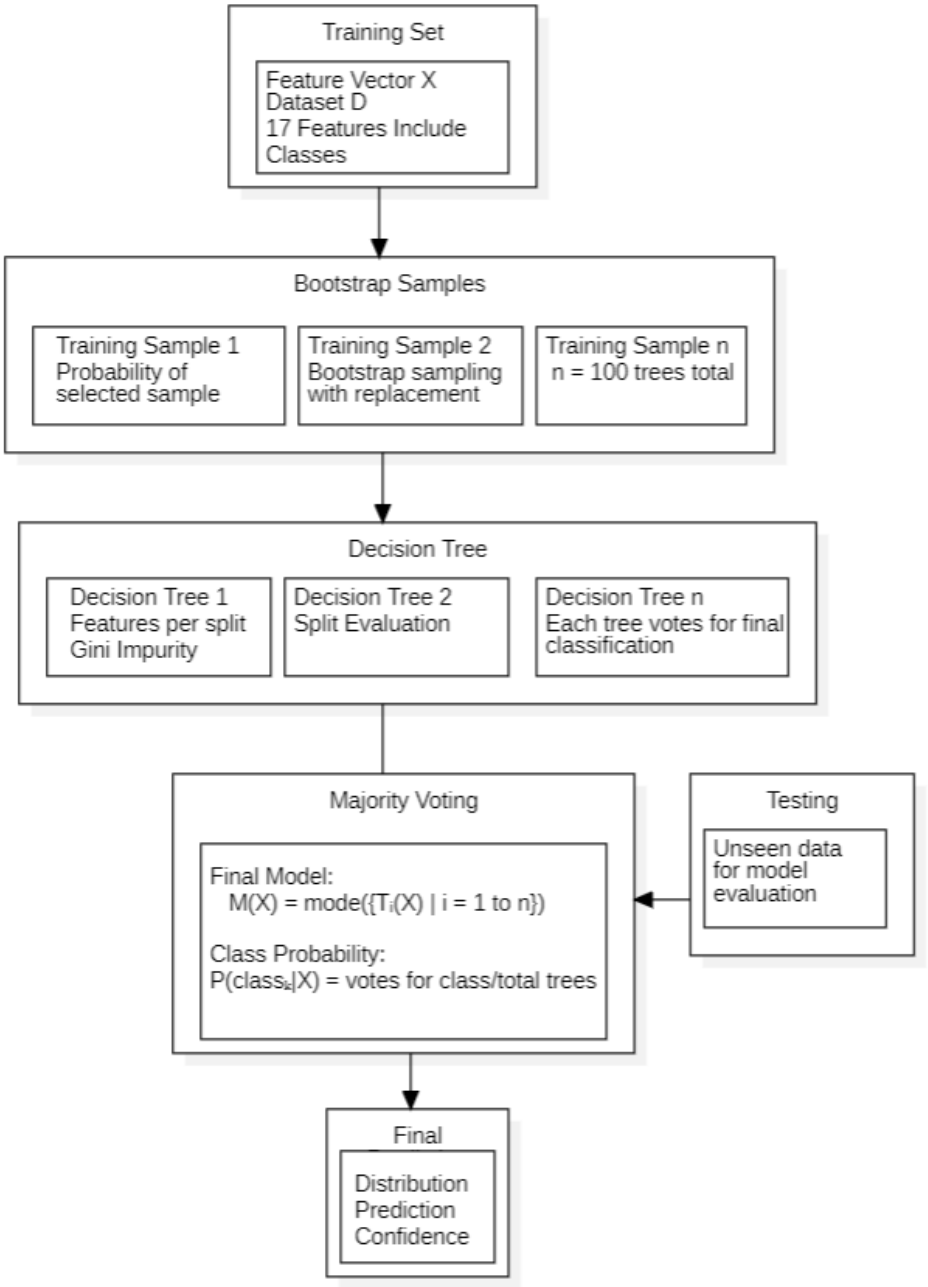


Figure 4.6: Model Architecture

Algorithm for Random Forest Model Training

Random Forest is a powerful ensemble learning method used for classification tasks by combining multiple decision trees. In our yoga pose classification project, we train the model using a dataset that includes 17 features, such as joint angles,

keypoint positions, and body alignments, to classify poses like Tree, Plank, Cobra, Warrior, Downward Dog, Goddess, and Mountain. Each row in the dataset consists of a feature vector

$$X = [f_1, f_2, \ldots, f_{17}] \tag{4.9}$$

where $f_1, f_2, \ldots, f_{17}$ represent numerical measurements of various pose characteristics. The target label y indicates the corresponding yoga pose. The dataset can be mathematically represented as:

$$D = \{(X_1, y_1), (X_2, y_2), \ldots, (X_n, y_n)\} \tag{4.10}$$

Before training the model, we performed certain configurations, which are summarized in Table 4.4.

Table 4.4: Training Parameters for the Model

Parameter	**Description**
Number of Decision Trees	100
Number of Features	17
Training Method	Bootstrap Sampling
Data Used per Tree	Approximately 63.2% of original samples
Purpose	Model uses 100 different decision trees to make predictions

The formula for the probability of a sample being included in a particular tree's training set is given by:

$$P(\text{selected}) = 1 - \left(1 - \frac{1}{n}\right)^n \approx 0.632 \tag{4.11}$$

This means that roughly 63.2% of the samples are used for each tree, while the remaining 36.8% are left out and are referred to as "out-of-bag" samples.

Each decision tree in the Random Forest uses the Gini Impurity criterion to evaluate splits at each node. Gini Impurity for a node N is calculated as:

$$G(N) = 1 - \sum_{i=1}^{C} p_i^2 \tag{4.12}$$

where p_i is the proportion of samples belonging to class i at node N, and C is the total number of classes (in this case, different yoga poses). For example, if a node contains 60% Tree pose and 40% Plank pose, the Gini Impurity is:

$$G(N) = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$$

A lower Gini Impurity value indicates a better split.

At each node, the algorithm randomly selects a subset of features to consider for the split, which is determined by the square root of the total number of features. Since our dataset contains 17 features, at each split, only $\sqrt{17} \approx 4$ features are randomly chosen for evaluation. This ensures that the model does not overfit by relying too heavily on any one feature.

The decision trees continue to grow by evaluating potential splits using the expanded Gini Index formula:

$$\begin{aligned} G_{\text{split}} = &\left(\frac{|D_{\text{left}}|}{|D_{\text{node}}|}\right) \times G_{\text{left}} \\ &+ \left(\frac{|D_{\text{right}}|}{|D_{\text{node}}|}\right) \times G_{\text{right}} \end{aligned} \tag{4.13}$$

where $|D_{\text{left}}|$ and $|D_{\text{right}}|$ are the number of samples in the left and right splits, respectively, and G_{left} and G_{right} are the Gini Impurities of the left and right child nodes. For instance, suppose a split results in the following:

Left node: 30 samples (20 Cobra, 10 Warrior) Right node: 20 samples (5 Cobra, 15 Warrior)

The Gini Impurities for the left and right nodes are:

$$\begin{aligned} G_{\text{left}} &= 1 - \left(\left(\frac{20}{30}\right)^2 + \left(\frac{10}{30}\right)^2\right) \\ &= 0.444 \end{aligned}$$

$$G_{\text{right}} = 1 - \left(\left(\frac{5}{20}\right)^2 + \left(\frac{15}{20}\right)^2\right)$$
$$= 0.375$$

The overall Gini score for the split is:

$$G_{\text{split}} = \left(\frac{30}{50} \times 0.444\right) + \left(\frac{20}{50} \times 0.375\right)$$
$$= 0.416$$

The tree then chooses the split with the lowest Gini score.

Once all trees are trained, the Random Forest makes a final prediction by majority voting. Each tree in the forest casts a vote for a class label, and the final prediction is the class with the most votes. If we consider the votes from 100 trees, where 40 trees vote for Tree pose, 35 for Plank pose, and 25 for Cobra pose, the final prediction will be Tree pose, as it has the majority. The probability of each class is given by:

$$P(\text{class}_k|X) = \frac{\text{number of votes for class}_k}{\text{total number of trees}} \tag{4.14}$$

For Tree pose, the probability would be $\frac{40}{100} = 0.4$, or 40%. The model's confidence in its prediction can also be computed as the maximum probability of any class, which in this case is 40% for Tree pose.

Finally, the Random Forest model is represented as:

$$M(X) = \text{mode}\left(\{T_i(X) \mid i = 1 \text{ to } n_{\text{estimators}}\}\right) \tag{4.15}$$

where $T_i(X)$ represents the prediction of the i-th tree for input X. This approach, combining many trees each with slightly different views of the data, ensures that the final model is robust, resistant to overfitting, and capable of accurately classifying yoga poses by considering different features such as arm, leg, and torso positions. The ensemble nature of Random Forest helps the model perform reliably, even in the presence of noisy or outlier data.

4.6.3 Feedback and Assistance

The feedback system provides real-time, actionable guidance to users based on their classified yoga pose, helping them to correct their posture effectively. The system begins by detecting keypoints corresponding to the user's body joints, normalizing these keypoints to ensure consistency when compared to predefined target poses stored within the system. To evaluate body posture, the system calculates the angles of various joints and compares them with the target angles.

Each joint is associated with a specific identifier, such as "left elbow," which is mapped in a dictionary for efficient comparison with the reference values. The system then checks if the calculated joint angles fall within an acceptable range, defined by a tolerance margin of ± 20 degrees from the target angle. This margin, referred to as the "error margin," accounts for natural variations in body movement and ensures that the system does not require unnecessary corrections. The ± 20 degree threshold strikes a balance between precision and flexibility, allowing the system to handle variations in posture while maintaining accuracy.

The system evaluates multiple joints simultaneously and generates feedback accordingly. If a joint's angle deviates beyond the error margin, the system adds a corrective suggestion to a feedback list. These suggestions are delivered to the user in both text and voice formats. Text feedback is provided when a joint's angle is outside the acceptable range, with specific guidance on whether the joint should be moved closer to or further away from the body part. Voice feedback serves as an additional layer of assistance, reinforcing the text suggestions in real time.

Ultimately, the system returns a list of suggestions, offering users clear, concise instructions on how to adjust their posture. This feedback mechanism ensures that users receive personalized, effective guidance, contributing to a safer and more efficient yoga practice.

4.6.3.1 Angle calculation:

The angle is calculated using vector mathematics, specifically through the dot product and arccosine. First of all, three points (P_1, P_2, P_3) are selected where the landmark at which the angle needs to be calculated is kept between the other two keypoints.

Suppose the system is calculating the left elbow angle using vector mathematics and keypoints:

Left shoulder (P_1): (3, 4),

Left elbow (P_2): (6, 8),

Left wrist (P_3): (9, 6)

Now the system calculates the vectors formed from the keypoints:

$$\text{Vector } \overrightarrow{P_1P_2} = [P_2[0] - P_1[0], P_2[1] - P_1[1]] = [6-3, 8-4] = [3, 4] \tag{4.16}$$

$$\text{Vector } \overrightarrow{P_2P_3} = [P_3[0] - P_2[0], P_3[1] - P_2[1]] = [9-6, 6-8] = [3, -2] \tag{4.17}$$

Now, the dot product of the two vectors is calculated as:

$$\text{Dot product} = \left(\text{Vector } \overrightarrow{P_1P_2}[0] \cdot \text{Vector } \overrightarrow{P_2P_3}[0]\right) + \left(\text{Vector } \overrightarrow{P_1P_2}[1] \cdot \text{Vector } \overrightarrow{P_2P_3}[1]\right) \tag{4.18}$$

$$= (3 \cdot 3) + (4 \cdot -2)$$

$$= 9 - 8$$

$$= 1$$

The magnitude (length) of a vector is:

$$\begin{aligned} \text{Magnitude } \overrightarrow{P_1P_2} &= \sqrt{(\text{Vector } \overrightarrow{P_1P_2}[0])^2 + (\text{Vector } \overrightarrow{P_1P_2}[1])^2} \\ &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= 5 \end{aligned} \tag{4.19}$$

$$\begin{aligned} \text{Magnitude } \overrightarrow{P_2P_3} &= \sqrt{(\text{Vector } \overrightarrow{P_2P_3}[0])^2 + (\text{Vector } \overrightarrow{P_2P_3}[1])^2} \\ &= \sqrt{3^2 + (-2)^2} \\ &= \sqrt{9 + 4} \\ &= \sqrt{13} \end{aligned} \tag{4.20}$$

Using the dot product and magnitudes, the cosine of the angle is:

$$\begin{aligned}\cos\theta &= \frac{\text{Dot product}}{\text{Magnitude } \overrightarrow{P_1P_2} \cdot \text{Magnitude } \overrightarrow{P_2P_3}} \\ &= \frac{1}{5 \cdot \sqrt{13}} \\ &= \frac{1}{18.05} \\ &\approx 0.0554\end{aligned} \tag{4.21}$$

The angle in radians is obtained using the arccosine:

$$\begin{aligned}\theta_{\text{radians}} &= \arccos(\text{clip}(\cos\theta, -1.0, 1.0)) \\ &= \arccos(0.0554) \\ &\approx 1.515 \text{ radians}\end{aligned} \tag{4.22}$$

Converting radians to degrees:

$$\begin{aligned}\theta_{\text{degrees}} &= \theta_{\text{radians}} \cdot \frac{180}{\pi} \\ &\approx 1.515 \cdot \frac{180}{\pi} \\ &\approx 86.85 \text{ degrees}\end{aligned} \tag{4.23}$$

To ensure the cosine value remains within a valid range:

$$\cos(\theta) = \text{clip}(\cos(\theta), -1.0, 1.0) \tag{4.24}$$

If the computed angle exceeds 180°, it is adjusted to:

$$\text{Angle} = 360^\circ - \text{Angle} \tag{4.25}$$

These formulas are applied for each joint's angle calculation, such as elbows, armpits, hips, knees, and ankles, using the relevant keypoints.
Following angles are calculated:

Table 4.5: Calculated Angle

Class		
Armpit	armpit_left	armpit_right
Elbow	left_elbow	right_elbow
Hip	left_hip	right_hip
Knee	left_knee	right_knee
Ankle	left_ankle	right_ankle

4.6.3.2 Mean Absolute Error(MAE) Calculation:

The Mean Absolute Error (MAE) is used to quantitatively assess how far a user's pose deviates from an ideal pose by comparing the joint angles of both poses. This metric gives a single scalar value that reflects the average deviation in degrees across all measured joints.

Required Inputs for MAE Calculation:

1. User's Pose Angles Vector:

$$\vec{A} = (a_1, a_2, \ldots, a_n)$$

   where a_k is the angle at the k^{th} joint measured from the live pose using vector mathematics (as described in Section 4.6.3.1).

2. Ideal Pose Angles Vector:

$$\vec{I} = (i_1, i_2, \ldots, i_n)$$

   where i_k is the reference angle at the k^{th} joint stored in the ideal dataset.

3. Number of Angles (Joints):

$$n = \text{Total number of joints considered for pose evaluation}$$

Steps to Calculate MAE:

1. Compute the Absolute Error per Joint:

   For each joint angle k, compute the absolute difference between the user's angle and the ideal angle:

$$d_k = |a_k - i_k| \tag{4.26}$$

   Where:

(a) d_k is the absolute error for joint k

(b) a_k is the angle from the user's pose

(c) i_k is the corresponding angle from the ideal pose

2. Sum of Absolute Errors:

   Sum all the absolute differences across the n joints:

$$\text{Total Error} = \sum_{k=1}^{n} d_k \tag{4.27}$$

3. Calculate Mean Absolute Error (MAE):

   Divide the total error by the number of joints to get the average error:

$$\text{MAE} = \frac{1}{n} \sum_{k=1}^{n} |a_k - i_k| \tag{4.28}$$

   This gives the Mean Absolute Error in degrees.

Suppose the user is performing the Tree Pose and the angles required are:
User's Extracted Angles (in degrees):

$\vec{A}$ = (178.645, 175.657, 133.263, 147.745, 97.564, 148.466, 178.625, 18.213, 134.255, 118.067)

Ideal Angles (in degrees):

$\vec{I}$ = (171.923, 179.882, 146.620, 157.399, 87.220, 155.133, 174.250, 25.560, 135.200, 105.920)

Step-by-step Absolute Error Calculation (using Equation 4.26):

1. Compute the Absolute Error per Joint

$$
\begin{aligned}
d_1 &= |178.645 - 171.923| = 6.722 \\
d_2 &= |175.657 - 179.882| = 4.225 \\
d_3 &= |133.263 - 146.620| = 13.357 \\
d_4 &= |147.745 - 157.399| = 9.654 \\
d_5 &= |97.564 - 87.220| = 10.344 \\
d_6 &= |148.466 - 155.133| = 6.667 \\
d_7 &= |178.625 - 174.250| = 4.375 \\
d_8 &= |18.213 - 25.560| = 7.347 \\
d_9 &= |134.255 - 135.200| = 0.945 \\
d_{10} &= |118.067 - 105.920| = 12.147
\end{aligned}
$$

2. Total Error

$$
\begin{aligned}
\text{Total Error} &= 6.722 + 4.225 + 13.357 + 9.654 + 10.344 + 6.667 + 4.375 + 7.347 + 0.945 \\
&\quad + 12.147 \\
&= 75.783
\end{aligned} \tag{4.29}
$$

3. MAE Calculation

$$
\text{MAE} = \frac{75.783}{10} = 7.578^\circ \tag{4.30}
$$

Final Result:

$$
\boxed{\text{Mean Absolute Error (MAE)} = 7.578^\circ}
$$

This result implies that, on average, each joint in the current Tree Pose is deviating by 7.578 degrees from the ideal posture.

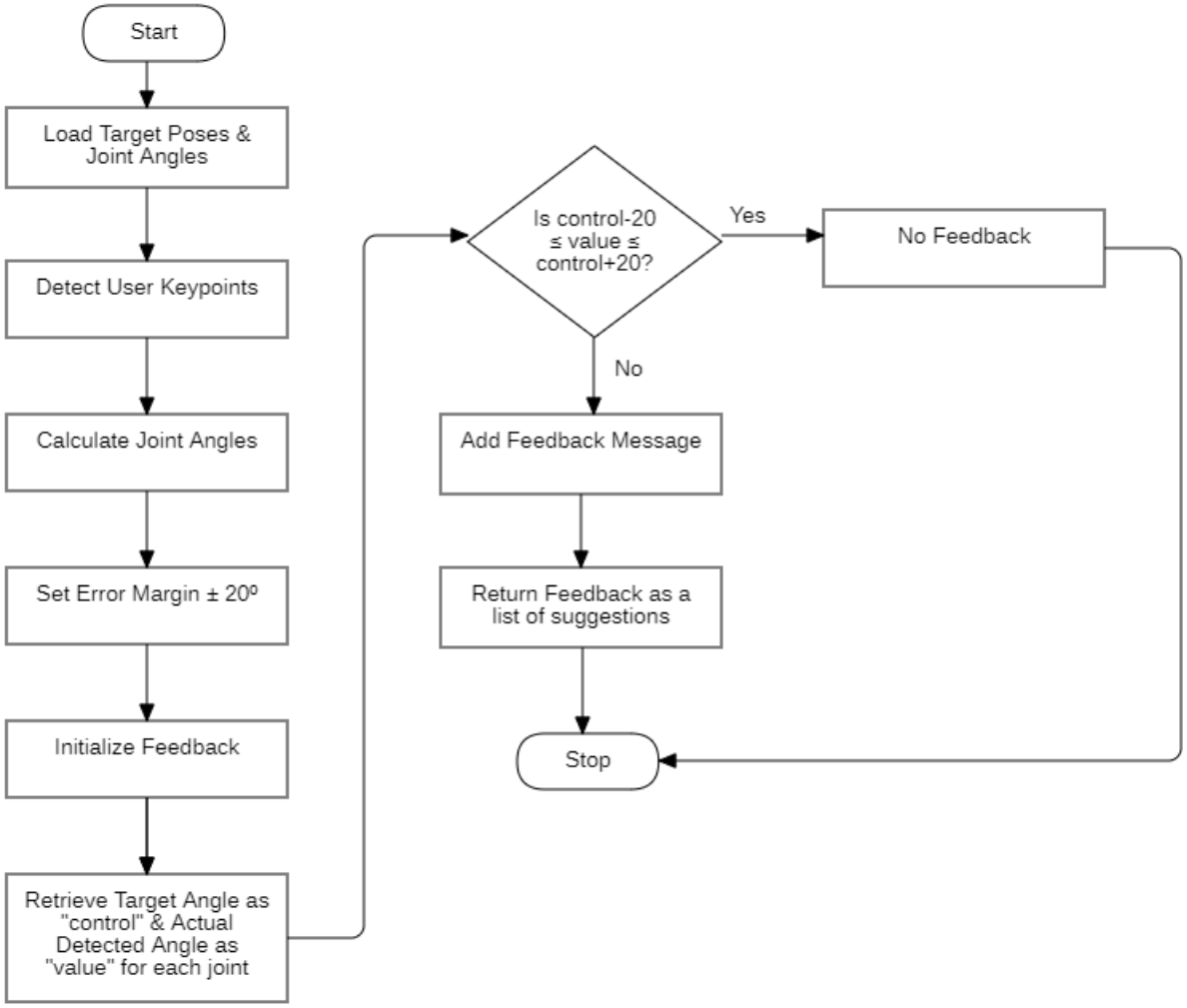


Figure 4.7: Flowchart of Feedback and Assistance

CHAPTER 5
RESULT AND ANALYSIS

Classification Report:

	precision	recall	f1-score	support
accuracy			0.96	11523
macro avg	0.96	0.95	0.96	11523
weighted avg	0.96	0.96	0.96	11523

The model demonstrates performance with an accuracy score of 96.08%, reflecting its ability to correctly classify the majority of instances. The precision, which ranges from 94% to 98%, indicates that the model is highly accurate in predicting the correct pose, while the recall, ranging from 92% to 99%, shows that it effectively captures most of the instances of the correct pose, minimizing false negatives. The F1-scores, which balance both precision and recall, range from 0.94 to 0.97, demonstrating that the model maintains a solid equilibrium between precision and recall, ensuring a well-rounded performance.

Despite the differences in the number of samples for each pose, the model consistently performs well across all poses. This consistency is reflected in both the macro average and the weighted average, both of which are 0.96. The macro average treats each pose equally, irrespective of the sample size, while the weighted average adjusts for class imbalances by giving more weight to poses with larger sample sizes. The fact that both averages are nearly identical suggests that the model handles varying sample sizes effectively and maintains balanced performance.

In conclusion, the model generalizes well to different poses and effectively handles variability in the data. The balanced performance across all metrics, combined with a final accuracy of 96.08%, indicates that the model is robust, reliable, and capable of managing both common and less frequent poses efficiently.

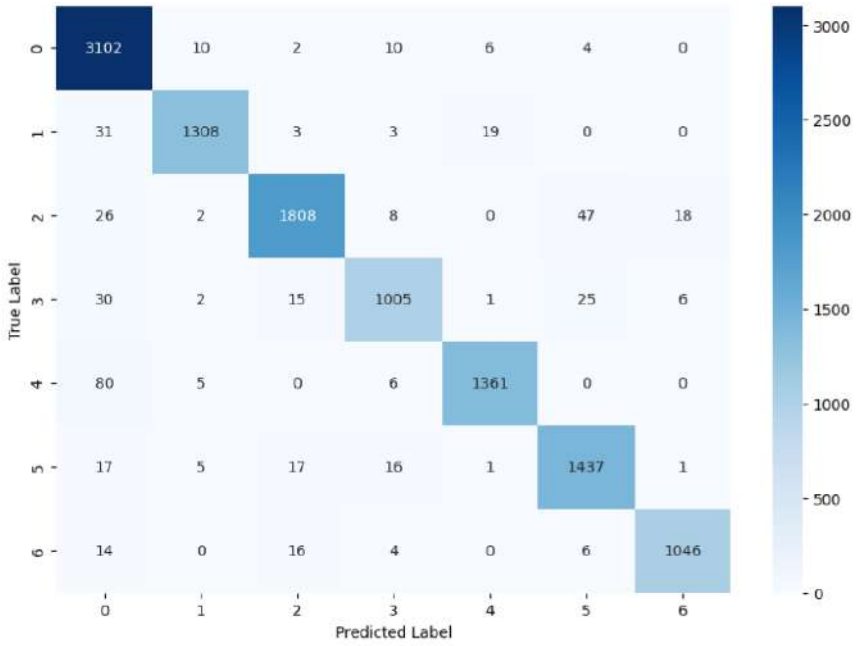


Figure 5.1: Confusion Matrix

The values in the confusion matrix are influenced by several factors related to the model, dataset, and classification process. High diagonal values indicate that the model has successfully learned distinguishing patterns, due to a well-represented dataset. The model correctly classifies most instances when the features of each class are distinct enough for clear separation. However, misclassifications (off-diagonal values) occur due to class similarity, where certain categories share overlapping features, making them harder to distinguish. For example, if two classes exhibit visual or contextual resemblance, the model may struggle to differentiate them accurately. Another major factor is data imbalance, where classes with fewer training samples tend to be misclassified more often, as the model has seen fewer examples of them during training. Overfitting and underfitting also contribute to errors—overfitting causes the model to memorize training data without generalizing well to new inputs, while underfitting prevents it from learning complex distinctions. Additionally, poor feature extraction leads the model to rely on weak or misleading characteristics, increasing confusion between similar classes. Noisy or mislabeled data further reinforce incorrect predictions, introducing inconsistencies that affect classification performance. These factors collectively shape the distribution of values in the confusion matrix, determining how well the model performs and where it encounters difficulties.

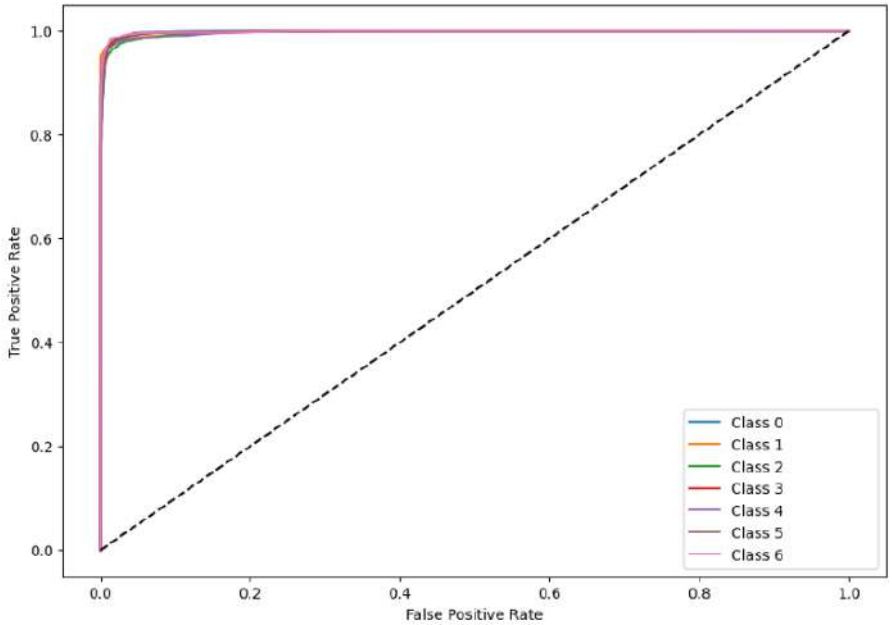


Figure 5.2: Receiver Operating Characteristics Graph

The figure presents the Receiver Operating Characteristic (ROC) curve, evaluating the performance of a classification model across multiple poses (Class 0 to Class 6). Each colored curve corresponds to a specific class, and the Area Under the Curve (AUC) for all classes ranges from 0.9 to 0.99. This indicates that the model achieves perfect classification accuracy, correctly distinguishing most of the poses. The model performs exceptionally well, with all curves reaching the top-left corner of the plot, where TPR is maximized and FPR is minimized. A maximized TPR means the model correctly identifies nearly all positive cases (high sensitivity), while a minimized FPR ensures it avoids most incorrect classifications (high specificity). The dashed diagonal line represents random guessing, and the fact that all curves are far above this line further confirms the model's excellent performance in classifying the pose data.

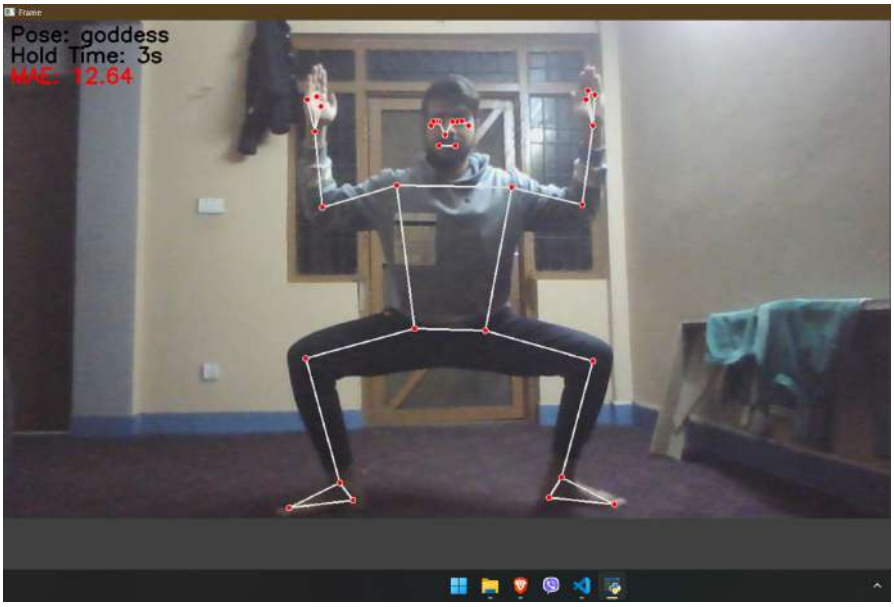


Figure 5.3: Pose Detection and Correction for Goddess

The camera captures a video feed of a person performing a pose. This video feed serves as the input to the system. Then the system detects the person in the frame and identifies key body landmarks (head, shoulders, elbows, wrists, hips, knees, and ankles). These landmarks are represented as red dots, and their connections form the pose skeleton visible in the image.

The detected landmarks are processed for consistency. This includes scaling and normalizing the positions relative to the frame dimensions and preparing them for analysis. Using the processed landmark data, the system classifies the current pose as "goddess." This label is displayed in the top-left corner of the image, indicating successful identification of the gesture.

Alongside the pose label, the system also displays the Hold Time, which shows how long the person has maintained the current pose—in this case, "3s" (3 seconds). This is useful for tracking pose endurance and stability. Additionally, the Mean Absolute Error (MAE) is shown to indicate how closely the user's pose matches the ideal version. In this instance, an MAE of 12.64 suggests there is some deviation from the optimal form. A lower MAE indicates better alignment with the reference pose.

The system visualizes the detected pose by overlaying the pose skeleton (red dots and connecting lines) on the video feed. This helps in real-time monitoring and

validation of the detected pose. Then the system calculates angles between joints to ensure the pose matches predefined criteria for the "goddess" pose. The pose is compared to a reference model for the "goddess" pose to validate its accuracy. This ensures the detected pose aligns with predefined standards. Based on the pose validation, the system generates feedback. Feedback could include corrections like "Bring your left arm closer to body" or "Move right leg away from body", helping the user improve their posture in real time.

Table 5.1: Comparision between RFC, SVM and CNN

Classifier	**Accuracy**	**Precision**	**Recall**	**F1 Score**
CNN	97.12%	97.14%	97.14%	97.14%
Random Forest	96.04%	96.63%	95.27%	96.14%
SVM	95.25%	95.27%	95.00%	95.27%

In our project, we trained and evaluated three models—Convolutional Neural Network (CNN), Random Forest Classifier (RFC), and Support Vector Machine (SVM)—using a dataset of body landmarks extracted from different yoga poses. Each model was assessed based on its accuracy, precision, recall, and F1 score. While CNN achieved the highest accuracy (97.12%), and SVM provided competitive performance (95.25% accuracy, 95.27% F1 Score), we ultimately chose Random Forest Classifier (RFC) for our system due to several key reasons.

Firstly, RFC demonstrated a balanced trade-off between accuracy (96.04%), precision (96.63%), and F1 Score (96.14%), indicating its strong generalization on structured numerical data like joint angles and body landmarks. Unlike CNN, which typically excels with image-based data, RFC effectively leveraged the structured input to make robust predictions without requiring extensive computational resources. Secondly, RFC was more interpretable and computationally efficient than CNN. Given that yoga pose correction relies on analyzing joint angles, decision trees in RFC allowed better transparency in understanding the model's

decision-making process. This interpretability made it easier to integrate real-time feedback and pose correction mechanisms based on angle deviations. Lastly, RFC performed consistently across different pose variations and lighting conditions, as it relied purely on numerical landmark data rather than image-based features. This ensured higher reliability in real-world applications, where environmental factors such as background clutter or lighting changes could affect CNN's performance.

Considering these factors, RFC was selected as the final model for our yoga pose detection and correction system, as it provided high accuracy, robustness, computational efficiency, and better adaptability to real-time feedback mechanisms.

CHAPTER 6
EPILOGUE

6.1 Conclusion

The Yoga Assistant system successfully detects and corrects yoga poses using computer vision and machine learning techniques. By leveraging MediaPipe for real-time keypoints detection and a Random Forest Classifier for pose classification, the system provides accurate posture analysis and helpful feedback. The project effectively bridges the gap in online yoga sessions by offering real-time guidance to users, allowing them to improve their practice without requiring a professional instructor. With an accuracy of 96.08%, the model performs well in classifying different yoga poses and delivering necessary corrections. This system is a step towards making yoga practice more accessible, safe, and effective for users of all levels.

Furthermore, this project highlights the potential of artificial intelligence in the health and wellness sector. The ability to detect poses, provide corrections, and improve user experience through real-time feedback showcases how technology can enhance traditional practices like yoga. The implementation of a cost-effective, easy-to-use, and scalable system ensures that individuals, regardless of location or skill level, can receive guidance to refine their yoga techniques. With further enhancements, this system could serve as a valuable tool for fitness enthusiasts, yoga instructors, and healthcare professionals aiming to promote better posture, flexibility, and overall well-being.

6.2 Limitations

While the Yoga Assistant system performs efficiently, it has some limitations:

6.2.1 Pose Variations

1. The system currently supports only seven yoga poses, limiting its usability for users practicing other poses.

6.2.2 Lighting and Background Conditions

1. The accuracy of pose detection may be affected by poor lighting or complex backgrounds.

6.2.3 Device Dependency

1. The system requires a good-quality camera and a capable computing device for smooth operation, making it less accessible for users with low-end devices.

6.2.4 Limited Feedback

1. While the system provides correction suggestions, it does not offer in-depth explanations or alternative exercises for users who struggle with a pose.

6.3 Future Enhancement

To improve and expand the functionality of the Yoga Assistant, the following enhancements can be considered:

6.3.1 Addition of More Poses

1. Expanding the dataset to include a wider variety of yoga poses to cater to more practitioners.

6.3.2 Personalized Feedback System

1. Implementing AI-driven adaptive feedback based on user flexibility, progress tracking, and injury prevention.

6.3.3 Multi-User Support

1. Allowing multiple users to practice yoga simultaneously with personalized feedback for each.

6.3.4 Real-Time Pose Correction Animation

1. Visualizing correct postures with animated guides for better understanding.

6.3.5 Integration with Wearable Devices

1. Connecting with smartwatches and fitness trackers to analyze additional data like heart rate and body posture stability.

6.3.6 Augmented Reality (AR) Assistance

1. Implementing AR overlays to guide users in achieving correct postures.

6.3.7 Community and Social Features

1. Adding options for users to share progress, join virtual yoga sessions, and receive feedback from peers or instructors.

6.3.8 Cloud-Based Performance Analytics

1. Storing user progress and pose history in the cloud for long-term tracking and improvement suggestions.

By implementing these improvements, the Yoga Assistant can become a more advanced and widely used tool for yoga practitioners, promoting a safer and more effective practice experience.

REFERENCES

[1] A. Garbett, Z. Degutyte, J. Hodge, and A. Astell, "Towards understanding people's experiences of ai computer vision fitness instructor apps," pp. 1619–1637, 06 2021.

[2] D. Kishore, S. Bindu, and M. Krishnamurthy, "Estimation of yoga postures using machine learning techniques," *International journal of yoga*, vol. 15, pp. 137–143, 09 2022.

[3] A. Mishra, D. Sahoo, I. Subhankar, and I. Samal, "Yogasiddhi: Ai-powered pose analysis using movenet for yoga refinement," *International Journal of Computer Applications*, vol. 186, pp. 33–39, 02 2024.

[4] A. Gupta and A. Jangid, "Yoga pose detection and validation," in *2021 International Symposium of Asian Control Association on Intelligent Robotics and Industrial Automation (IRIA)*, pp. 319–324, 2021.

[5] K. Aarthy and A. Nithya, "Automated yoga pose recognition using enhanced chicken swarm optimization with deep learning," *Multimedia Tools and Applications*, vol. 83, pp. 86299–86321, 10 2024.

[6] D. M. Kishore, S. Bindu, and N. K. Manjunath, "Estimation of yoga postures using machine learning techniques," *International Journal of Yoga*, vol. 15, no. 2, pp. 137–143, 2022.

[7] Y. Agrawal, Y. Shah, and A. Sharma, "Implementation of machine learning technique for identification of yoga poses," in *2020 IEEE 9th International Conference on Communication Systems and Network Technologies (CSNT)*, pp. 40–43, 2020.

[8] R. Jadhav, V. Ligde, R. Malpani, P. Mane, and S. Borkar, "Aasna: Kinematic yoga posture detection and correction system using cnn," *ITM Web of Conferences*, vol. 56, 08 2023.

[9] A. K. Rajendran and S. C. Sethuraman, "A survey on yogic posture recognition," *IEEE Access*, vol. 11, pp. 11183–11223, 2023.

[10] P. Kulkarni, S. Gawai, S. Bhabad, A. Patil, and S. Choudhari, "Yoga pose recognition using deep learning," 03 2024.

[11] S. Negi, M. Garg, H. Maindola, V. Kansal, U. Jain, and S. Bhatla, "Real-time human pose estimation: A mediapipe and python approach for 3d detection and classification," pp. 128–133, 11 2023.

[12] Wikipedia contributors, "HTML," 2024. [Online; accessed February 9, 2025].

[13] Wikipedia contributors, "Semantic HTML," 2024. [Online; accessed February 9, 2025].

[14] MDN contributors, "What is CSS?," 2024. [Online; accessed February 9, 2025].

[15] MDN contributors, "JavaScript," 2024. [Online; accessed February 9, 2025].

[16] Django Software Foundation, "Django: The web framework for perfectionists with deadlines," 2025. [Online; accessed February 9, 2025].

[17] A. Ibrahim, *Data Science using Python Language with Applications*. 03 2024.

[18] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, *et al.*, "Array programming with numpy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[19] T. pandas development team, *pandas: Python Data Analysis Library*. Accessed: 2025-02-09.

[20] O. Team, *OpenCV: Open Source Computer Vision Library*. Accessed: 2025-02-09.

[21] Google, "MediaPipe," 2025. [Online; accessed February 9, 2025].

[22] scikit-learn developers, *RandomForestClassifier — scikit-learn 1.6.1 documentation*. Accessed: 2025-02-09.

[23] P. S. Foundation, *pickle — Python object serialization*. Accessed: 2025-02-09.

[24] N. M. Bhat, *pyttsx3: Text to Speech (TTS) library for Python 3.* Accessed: 2025-02-09.

[25] P. S. Foundation, *multiprocessing — Process-based parallelism.* Accessed: 2025-02-09.

APPENDIX A
APPENDIX

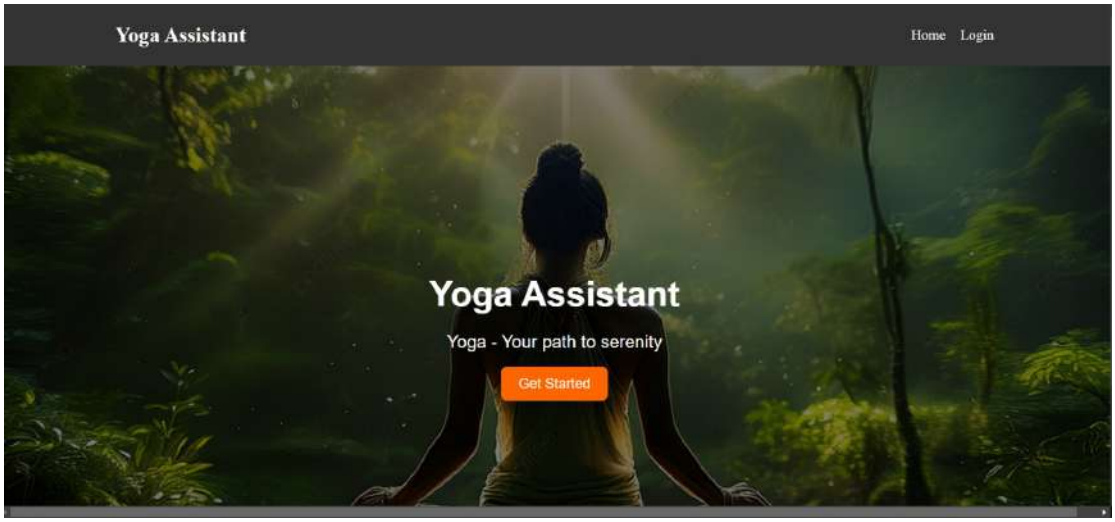


Figure A.1: Home Page

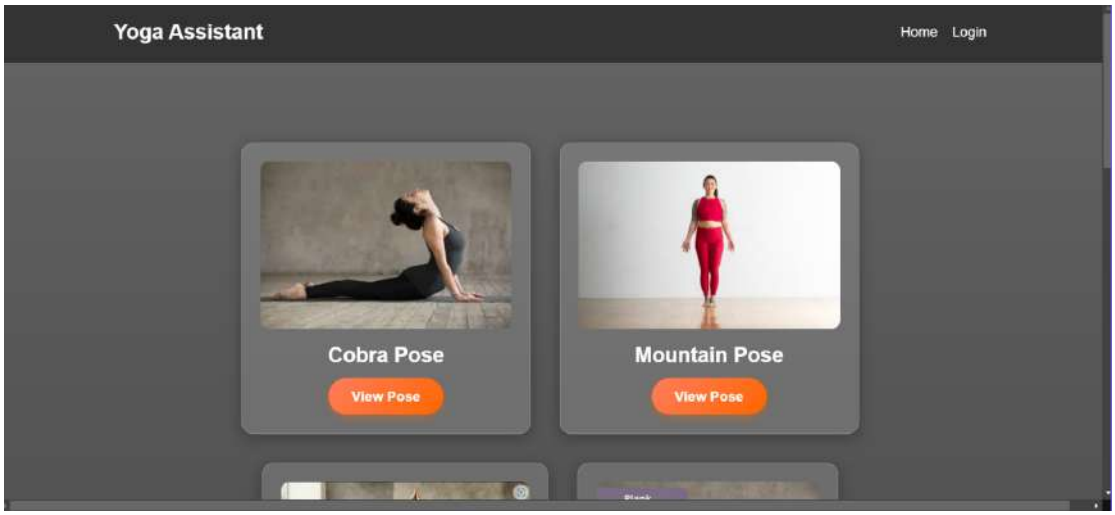
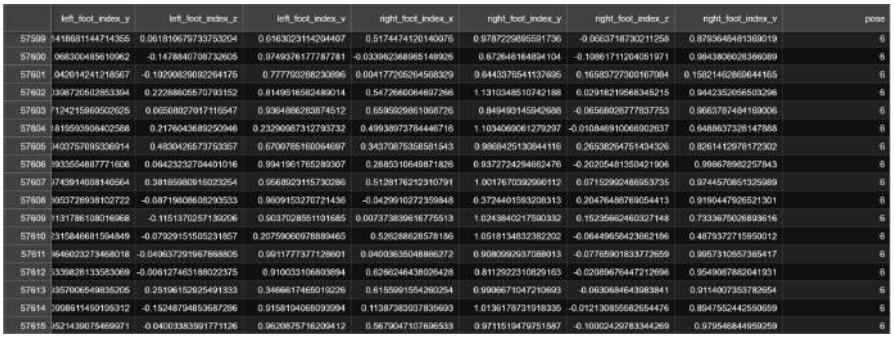
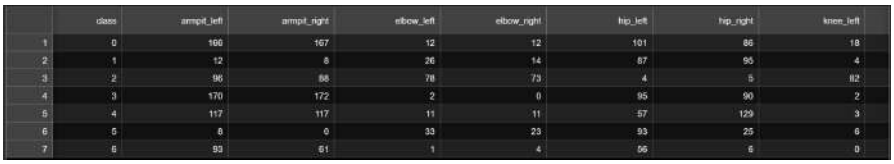


Figure A.2: Pose Selection



	left_foot_index_y	left_foot_index_z	left_foot_index_v	right_foot_index_x	right_foot_index_y	right_foot_index_z	right_foot_index_v	pose
57599	1418681144714355	0.06181067973373204	0.6163023114294407	0.5174474120140076	0.9787229895591736	-0.0663718730211258	0.8793646481369019	6
57600	068300485610962	-0.1478840708732605	0.9749376177787781	-0.033982388965148926	0.672646164894104	-0.10861711264051971	0.9843806028366089	6
57601	042614241218567	-0.10290829092264175	0.777793288230896	0.004177205264568329	0.6443376541137695	0.16583727300167084	0.15821462869644165	6
57602	3367205025853394	0.22288605570793152	0.8149516582489014	0.5472660064697266	1.1310348510742188	0.02916219568345215	0.9442352056503296	6
57603	1242158965502625	0.06508027917116547	0.9364866256874512	0.6595929861068726	0.8494931459426886	-0.06568926777837753	0.9663787484169006	6
57604	1819593806402588	0.21760436892509460	0.23290987312793732	0.49938973784446716	1.1034069061279297	-0.010846910068902637	0.6488637328147888	6
57605	1403757695336914	0.4830426573753357	0.6700785160064697	0.34370875358581543	0.9868425130844116	0.26538264751434326	0.8261412978172302	6
57606	1933554887771606	0.09423232704401016	0.9941961765289307	0.2885310649871826	0.9372724294662476	-0.20205481350421906	0.99867892257843	6
57607	1743914008140564	0.38185980616023254	0.9568923115730286	0.5128176212310791	1.0017670392990112	0.07152992486953735	0.9744570851325989	6
57608	1953726938102722	-0.08719806868293533	0.9909153270721436	-0.042991027235984	0.3724401569208313	0.20476486769054413	0.9190447926521301	6
57609	1131786108016968	-0.11513702571392	0.9037028551101685	0.007373839616775513	1.0243840217590332	0.15235662460327148	0.7333675026893616	6
57610	2315846681594849	-0.07929151505231857	0.20759060978889465	0.526288628578186	1.0518134832382202	-0.06449658423662186	0.4879372715950012	6
57611	1646023273468018	-0.04963729919678688	0.9911777377128601	0.04009636048888272	0.9080992937088013	-0.07765901833772659	0.9957310557365417	6
57612	1339828133583069	-0.008127463188022375	0.9100331068038994	0.6266246438026428	0.8112922310829163	-0.02089676447212696	0.9549087882041931	6
57613	1357006549835205	0.25196152925491333	0.3466617465019226	0.6155991554260254	0.9906671047210693	-0.06306846439838841	0.9114007353782654	6
57614	1998611459195312	-0.15248794853687286	0.9158194065093994	0.11387383937835693	1.0136178731918335	-0.012130855682654476	0.8947552442550659	6
57615	1521439075469971	-0.04003383591771126	0.9620857516209412	0.5679047107696533	0.9711519479751587	-0.10002429783344269	0.979546844959259	6

Figure A.3: Data Pose CSV



	class	armpit_left	armpit_right	elbow_left	elbow_right	hip_left	hip_right	knee_left
1	0	166	167	12	12	101	86	18
2	1	12	8	26	14	87	95	4
3	2	96	88	78	73	4	5	82
4	3	170	172	2	0	95	90	2
5	4	117	117	11	11	57	129	3
6	5	8	0	33	23	93	25	6
7	6	93	61	1	4	56	6	0

Figure A.4: Data Angle CSV

Class	Precision	Recall	F1-Score	Support
0	0.94	0.99	0.96	3134
1	0.98	0.96	0.97	1364
2	0.97	0.95	0.96	1909
3	0.96	0.93	0.94	1084
4	0.98	0.94	0.96	1452
5	0.95	0.96	0.95	1494
6	0.98	0.96	0.97	1086
Accuracy	0.96			
Macro Avg	0.96	0.95	0.96	11523
Weighted Avg	0.96	0.96	0.96	11523

Figure A.5: Classification Report of Each Pose

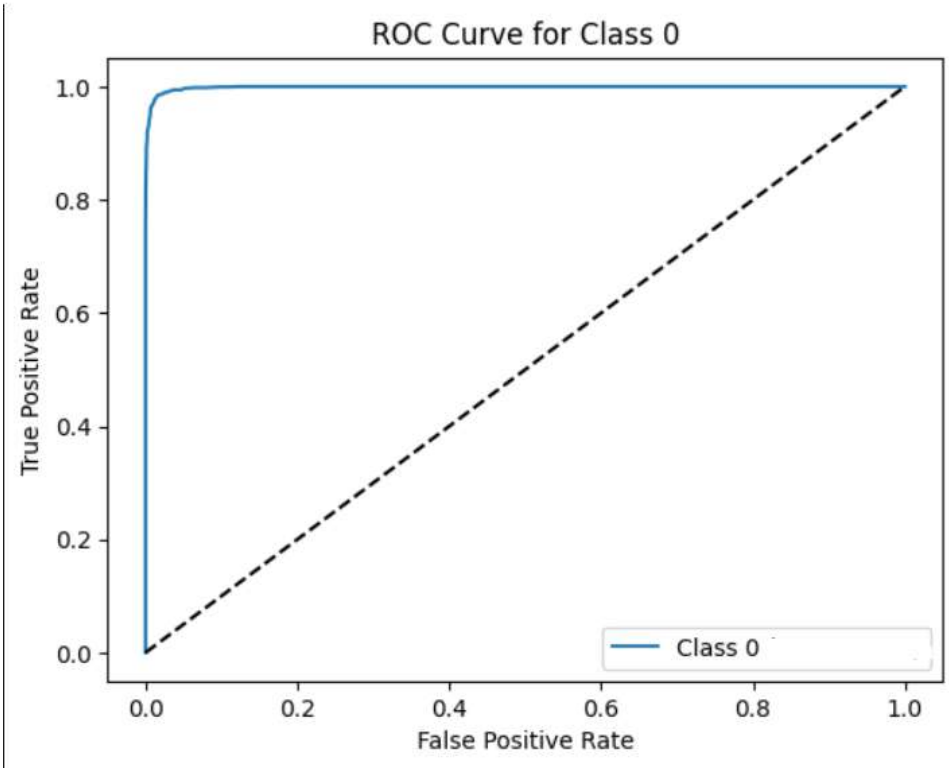


Figure A.6: ROC Curve for Class 0

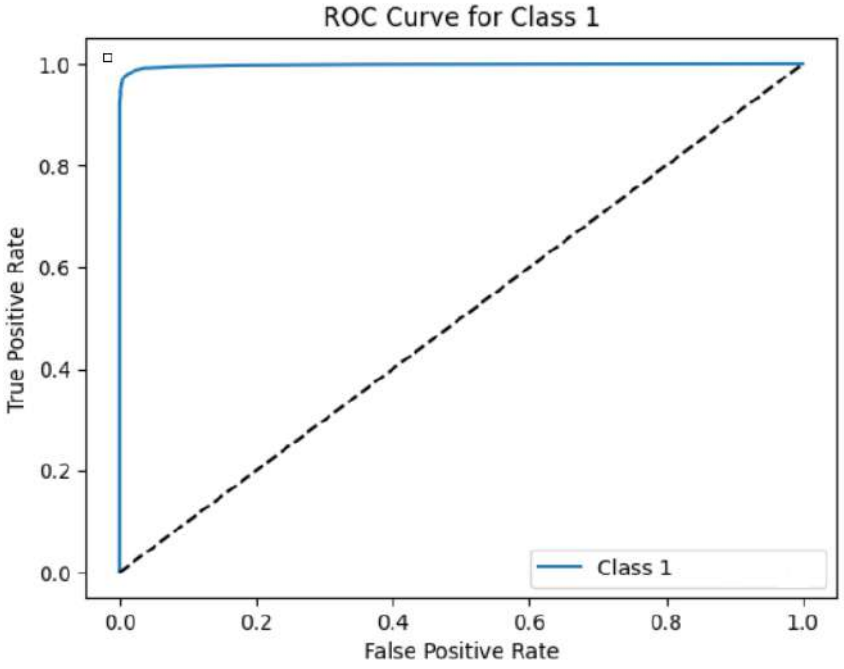


Figure A.7: ROC Curve for Class 1

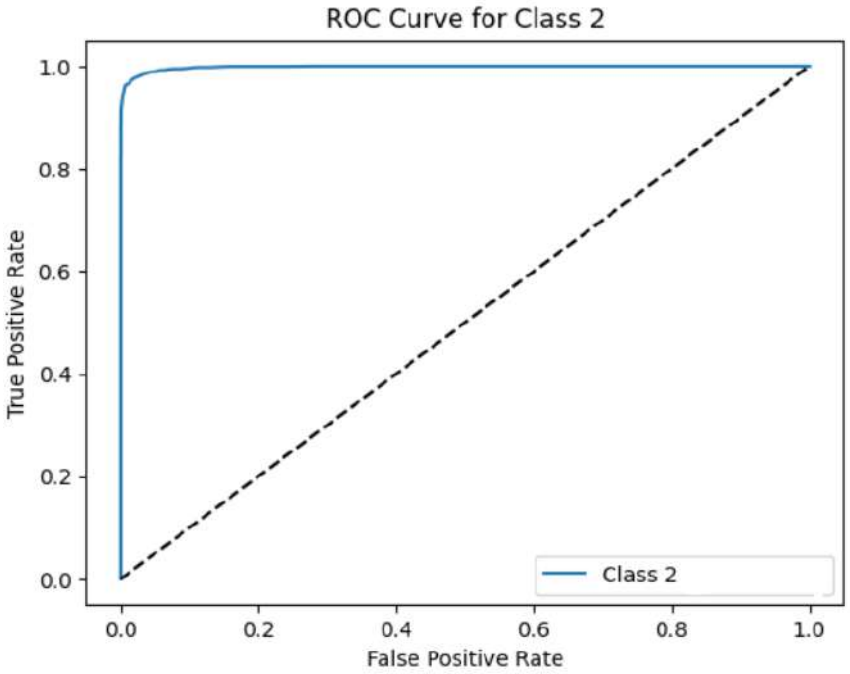


Figure A.8: ROC Curve for Class 2

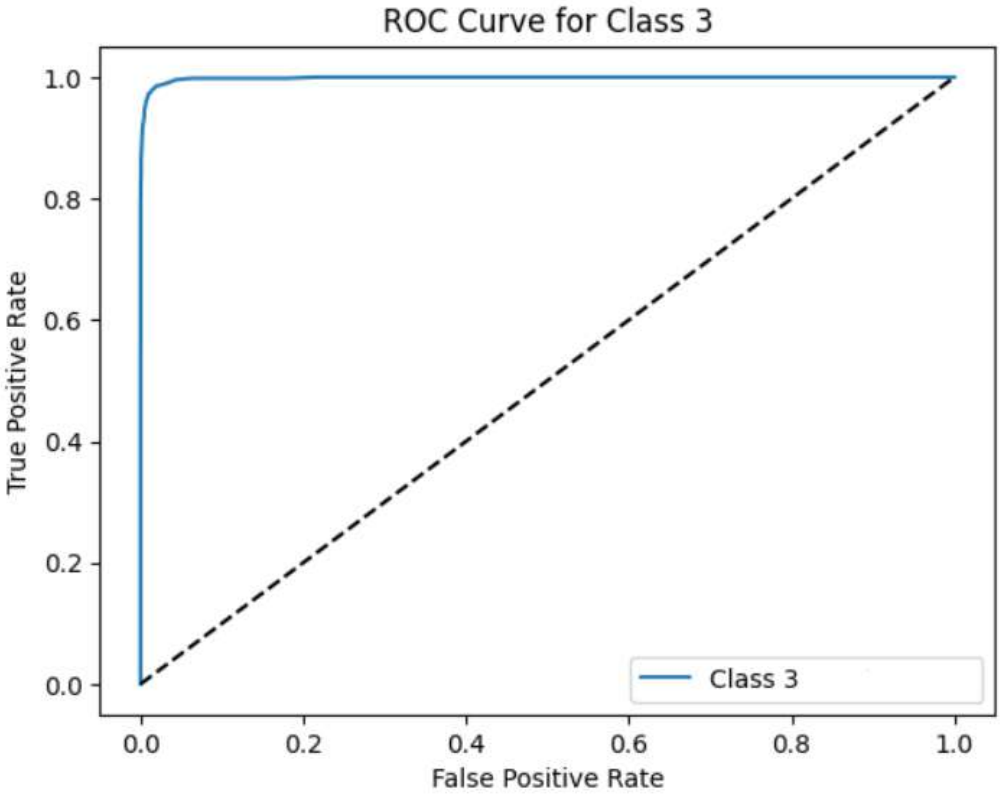


Figure A.9: ROC Curve for Class 3

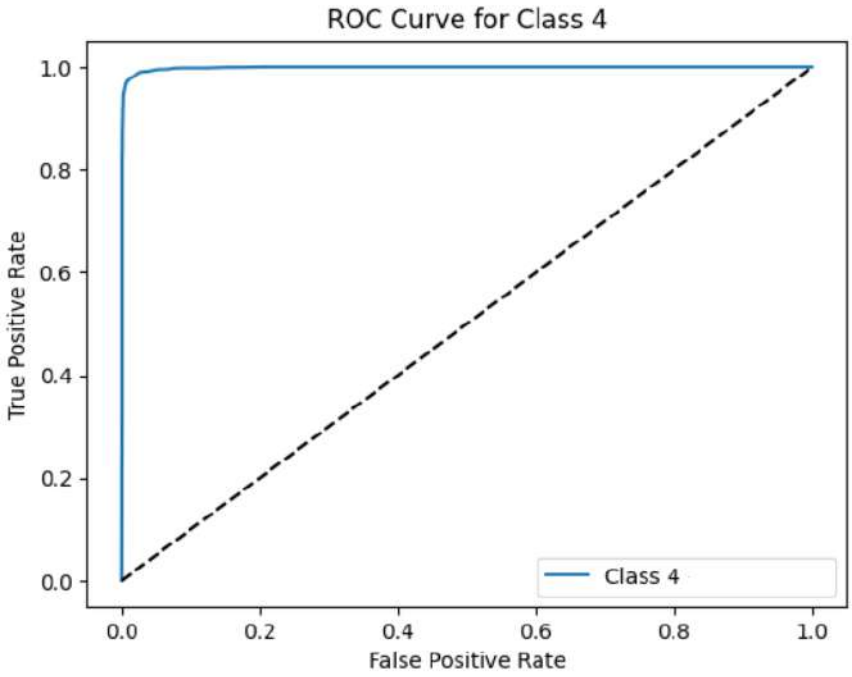


Figure A.10: ROC Curve for Class 4

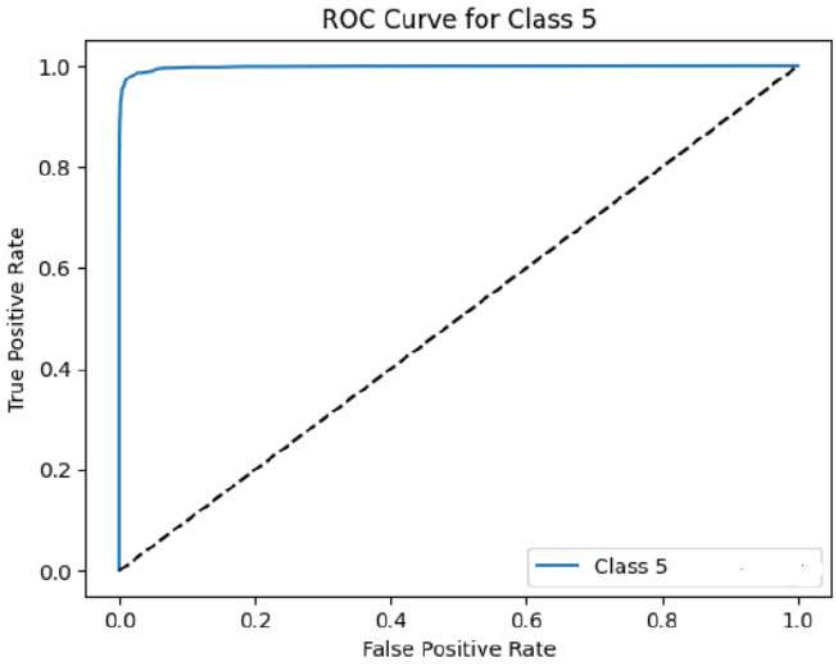


Figure A.11: ROC Curve for Class 5

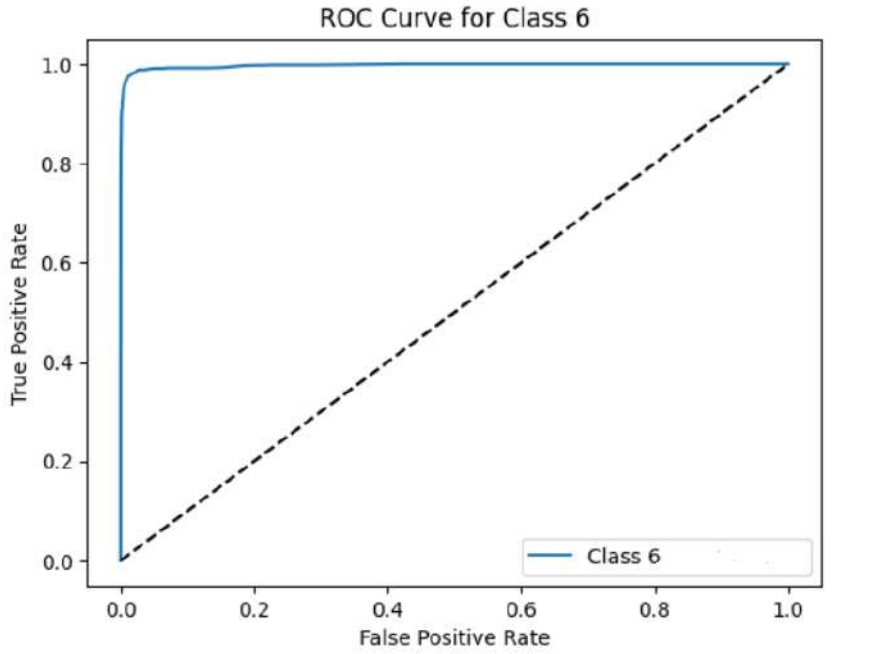


Figure A.12: ROC Curve for Class 6

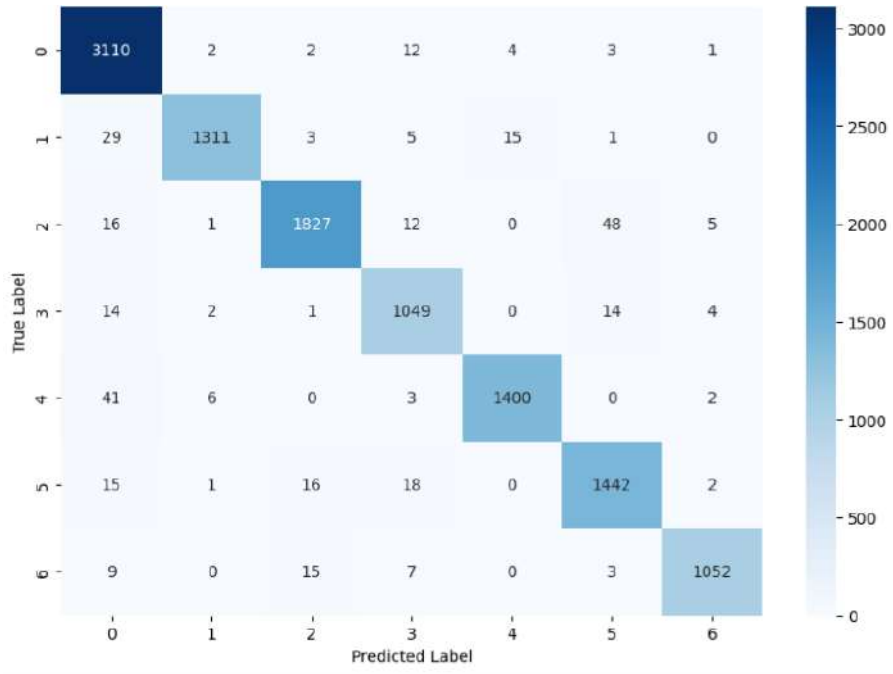


Figure A.13: Confusion matrix for CNN model

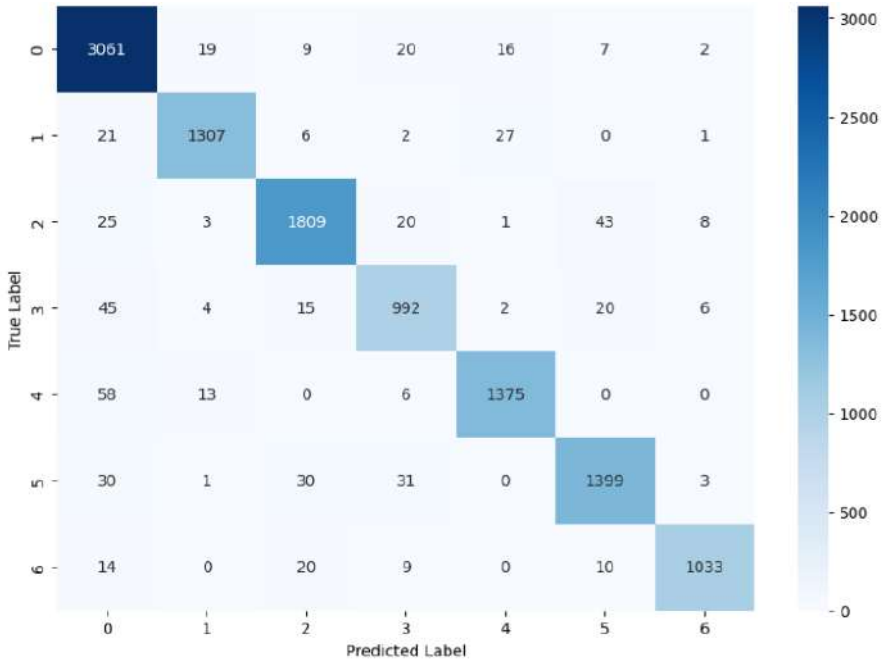


Figure A.14: Confusion matrix for SVM model

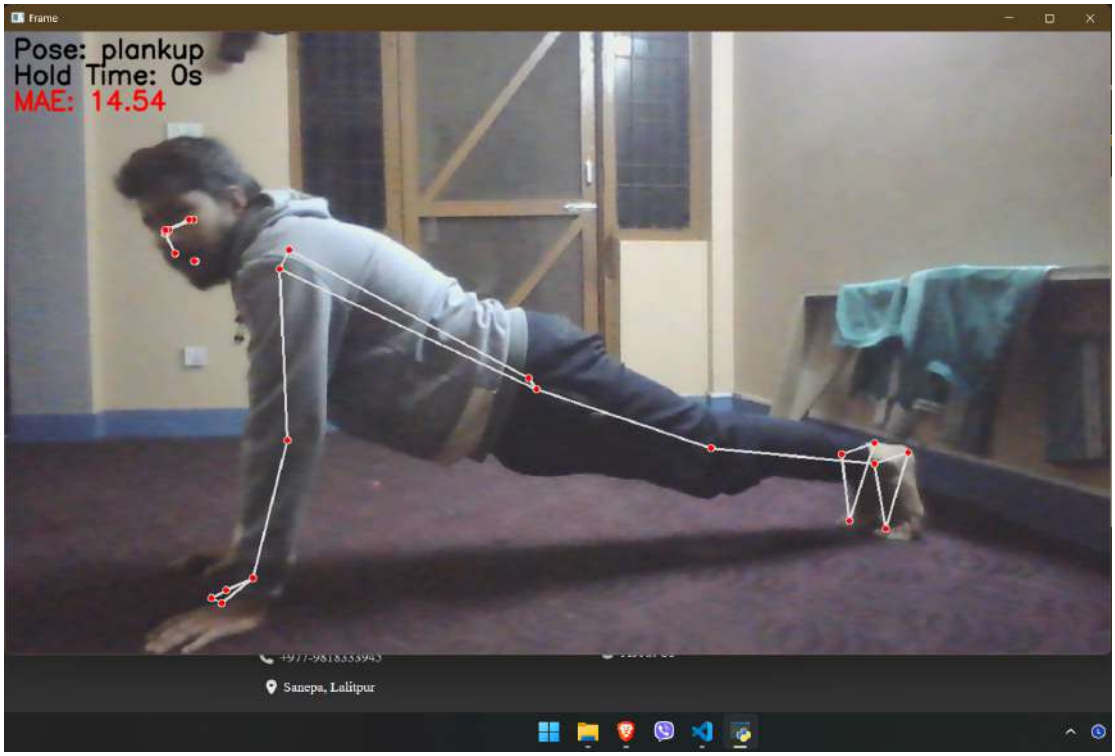


Figure A.15: Pose Detection and Correction for PlankUp

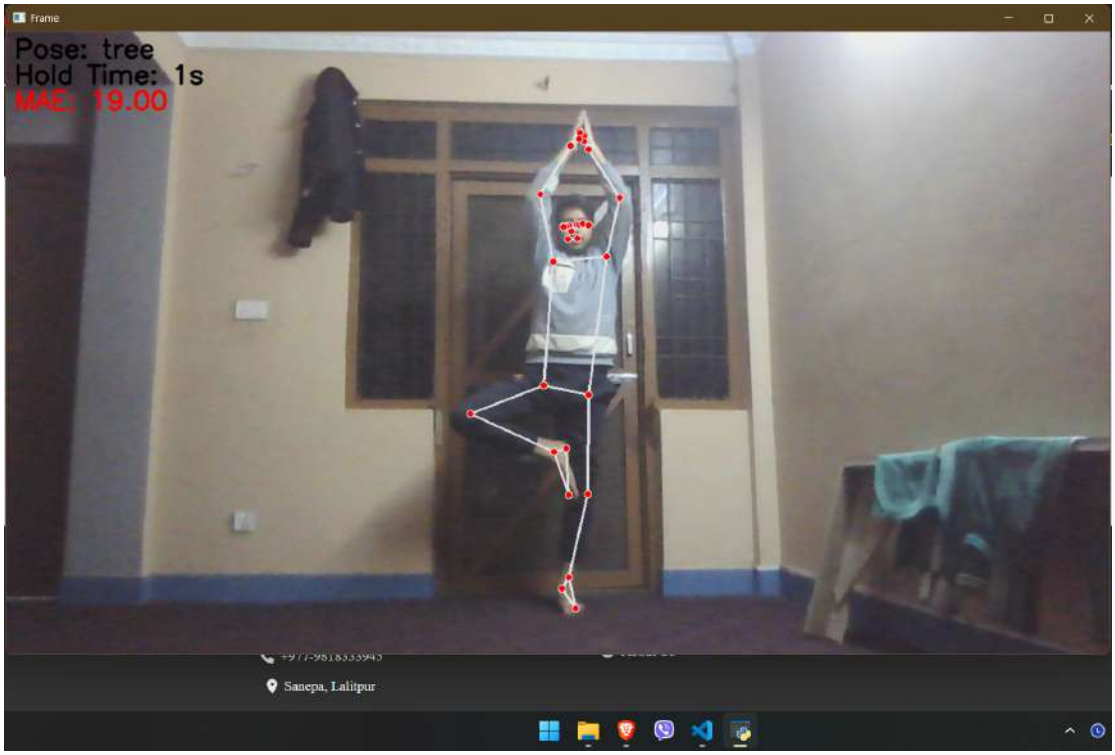


Figure A.16: Pose Detection and Correction for Tree

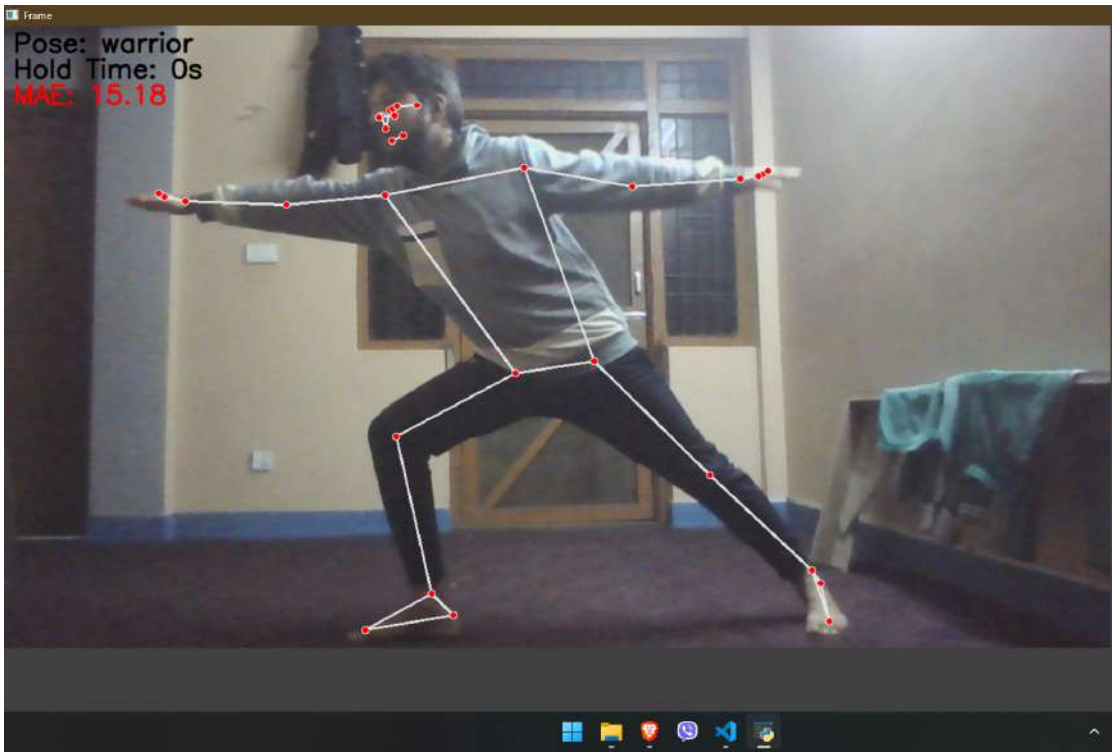


Figure A.17: Pose Detection and Correction for Warrior

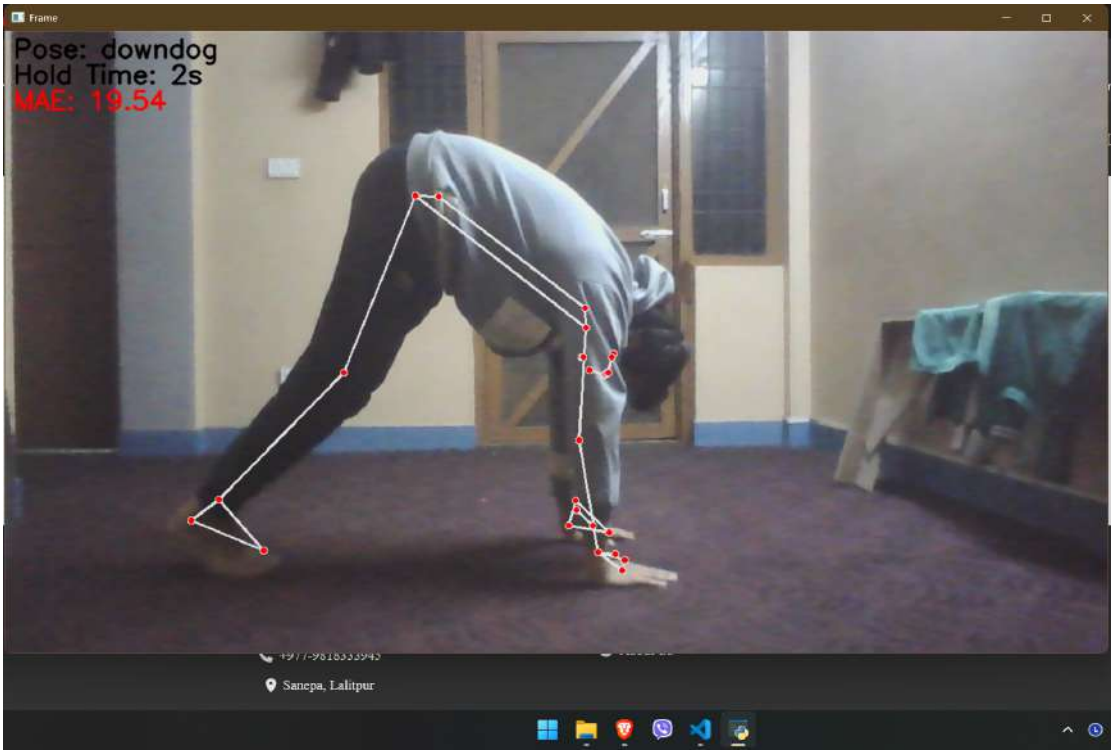


Figure A.18: Pose Detection and Correction for Downward dog

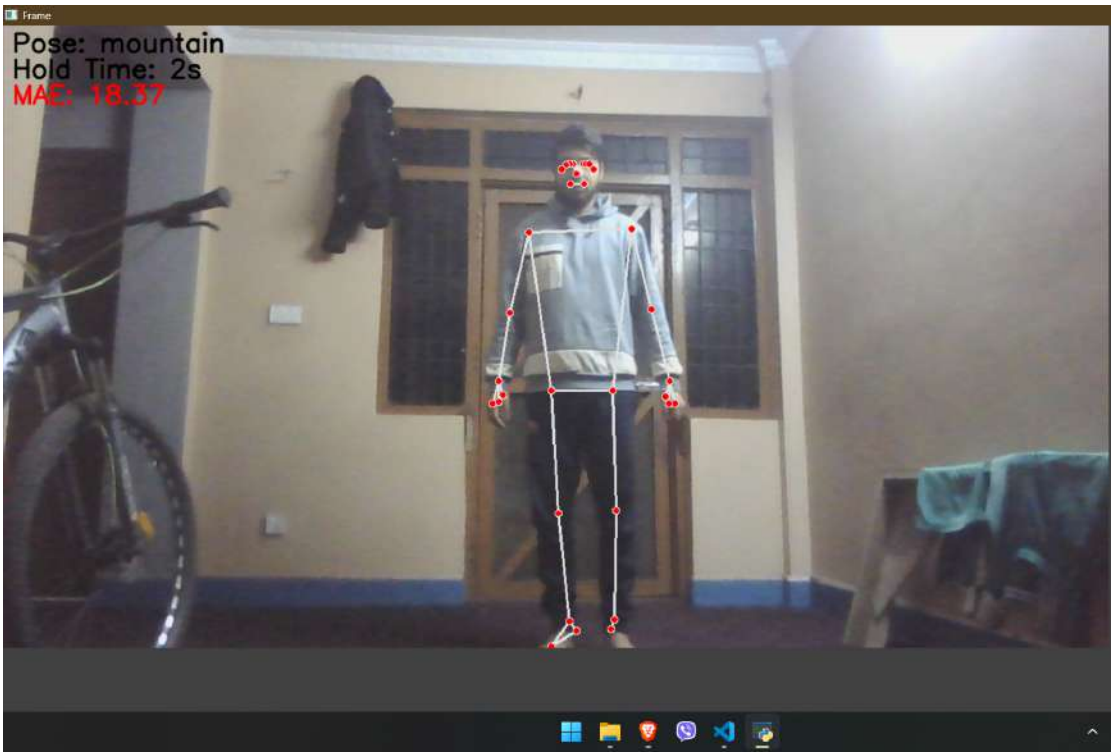


Figure A.19: Pose Detection and Correction for Mountain

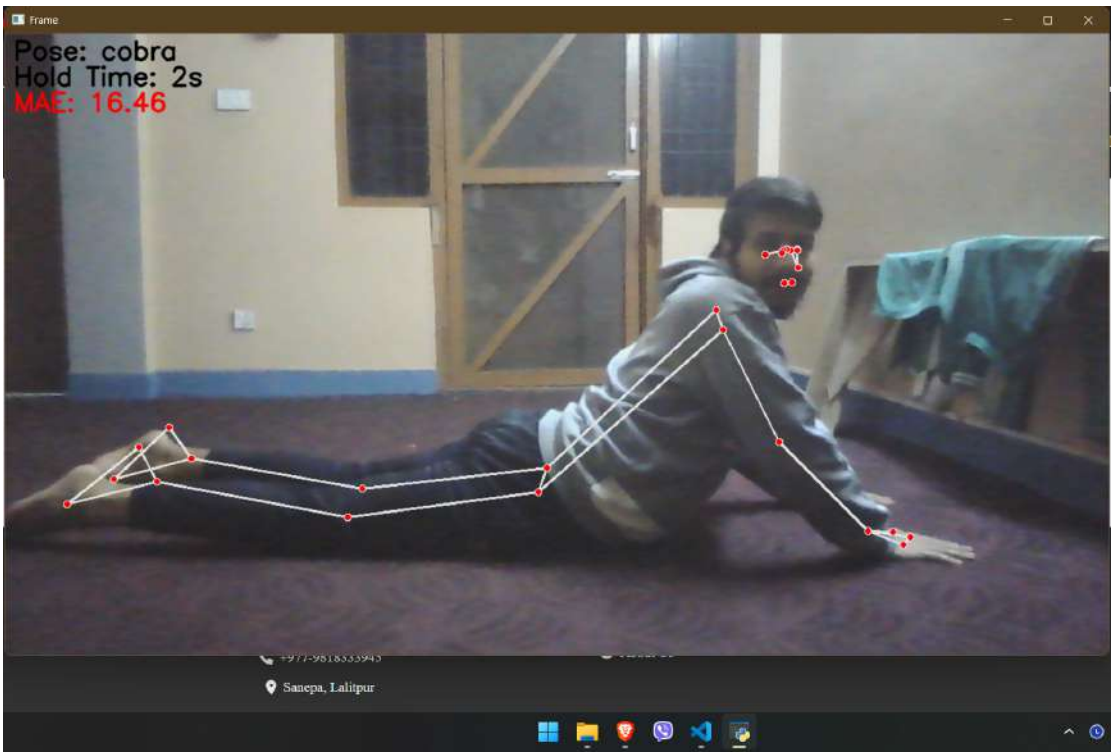


Figure A.20: Pose Detection and Correction for Cobra